

A Comprehensive Survey on Agent Skills: Taxonomy, Techniques, and Applications

Yingli Zhou, Shu Wang, Yaodong Su, Wenchuan Du, Yixiang Fang *Member, IEEE*, Xuemin Lin *Fellow, IEEE*
 The Chinese University of Hong Kong, Shenzhen
 yinglizhou@link.cuhk.edu.cn; shuwang3@link.cuhk.edu.cn; yaodongsu@link.cuhk.edu.cn; doorvant@gmail.com;
 fangyixiang@cuhk.edu.cn; linxuemin@cuhk.edu.cn;

Abstract—Large language model (LLM)-based agents that reason, plan, and act through tools, memory, and structured interaction are emerging as a promising paradigm for automating complex workflows. Recent systems such as OpenClaw and Claude Code exemplify a broader shift from passive response generation to action-oriented task execution. Yet as agents move toward open-ended, real-world deployment, relying on from-scratch reasoning and low-level tool calls for every task become increasingly inefficient, error-prone, and hard to maintain. This survey examines this challenge through the lens of *agent skills*, which we define as reusable procedural artifacts that coordinate tools, memory, and runtime context under task-specific constraints. Under this view, agents and skills play complementary roles: agents handle high-level reasoning and planning, while skills form the operational layer that enables reliable, reusable, and composable execution. Skills are therefore central to the scalability, robustness, and maintainability of modern agent systems. We organize the literature around four stages of the agent skill lifecycle—representation, acquisition, retrieval, and evolution—and review representative methods, ecosystem resources, and application settings across each stage. We conclude by discussing open challenges in quality control, interoperability, safe updating, and long-term capability management. All related resources, including research papers, open-source data, and projects, are collected for the community in <https://github.com/JayLZhou/Awesome-Agent-Skills>.

I. INTRODUCTION

Large language model (LLM)-based agents are emerging as a powerful paradigm for automating complex tasks. Fundamentally, an LLM-based agent is an autonomous system that leverages an LLM as its cognitive engine to perceive its environment, interpret task context, reason over abstract goals, and execute actions through planning, tool use, memory retrieval, and structured interaction [1]–[6]. Recent pioneering systems, such as OpenClaw [7] Manus [8], and Claude Code [9], vividly exemplify this paradigm, marking a broader transition in intelligent systems from passive response generation to proactive, action-oriented task execution.

As LLM-based agents are deployed in a growing range of scenarios and entrusted with increasingly complex tasks, tool augmentation has become a central design principle, enabled by APIs, plugins, and protocol layers such as MCP [10], [11]. However, practical experience shows that mere access to tools does not determine *when* a capability should be invoked, *how* multiple tools should be coordinated, *how* failures should be handled, or *how* outputs should be validated. As tasks become more long-horizon and heterogeneous, relying on an agent

to deduce these procedural steps from scratch for every task leads to severe brittleness, high latency, and unreliability. This “procedural gap” has emerged as a major bottleneck.

This gap motivates a fundamental shift toward a skill-centric view of agent systems. In this survey, we define *agent skills* as reusable procedural artifacts that encode the specific “how-to” knowledge for coordinating tools, memory, and runtime context under concrete constraints [12]–[14]. Within this framework, agents and skills form a highly synergistic *hierarchical relationship*: the agent acts as the high-level cognitive planner responsible for intent interpretation and goal decomposition, while skills constitute the vital operational layer that translates these abstract plans into robust low-level execution. The importance of skills lies in their role as the agent’s “muscle memory.” By externalizing procedural know-how into reusable artifacts, skills empower agents to bypass redundant step-by-step reasoning, drastically reduce execution errors, and transform transient actions into persistent capabilities that are easily retrieved, composed, revised, and governed across repeated tasks.

More broadly, accumulating experience into reusable skills is a long-standing pattern in human learning. People do not solve every task from scratch; they progressively convert repeated practice, demonstrations, failures, and expert instruction into reusable procedures. As illustrated in Fig. 1, this externalization process can be viewed as a long trajectory from embodied craft knowledge, to codified engineering procedures, to digital tools and programmable workflows, and now to agent-native skill ecosystems.

Against this backdrop, the recent LLM era has turned skill-centric agents into a fast-growing research area. Fig. 2 shows the rapid growth of representative papers from April 2023 to April 2026. At the same time, progress is fragmented across multiple threads: skill acquisition from human expertise, traces, tasks, and corpora [12]–[14]; retrieval from large and heterogeneous skill libraries [15]–[17]; runtime selection and composition under state and budget constraints [5], [6], [18]; and post-deployment revision, evolution, and governance [19], [20]. This fragmentation motivates a focused and systematic survey of agent skills and skill-centric agent ecosystems.

To address this need, this survey provides a focused and systematic review of agent skills and skill-centric LLM agent ecosystems. We organize the literature around four lifecycle modules: skill representation, skill acquisition, skill retrieval, and skill evolution. Beyond method categorization, we review

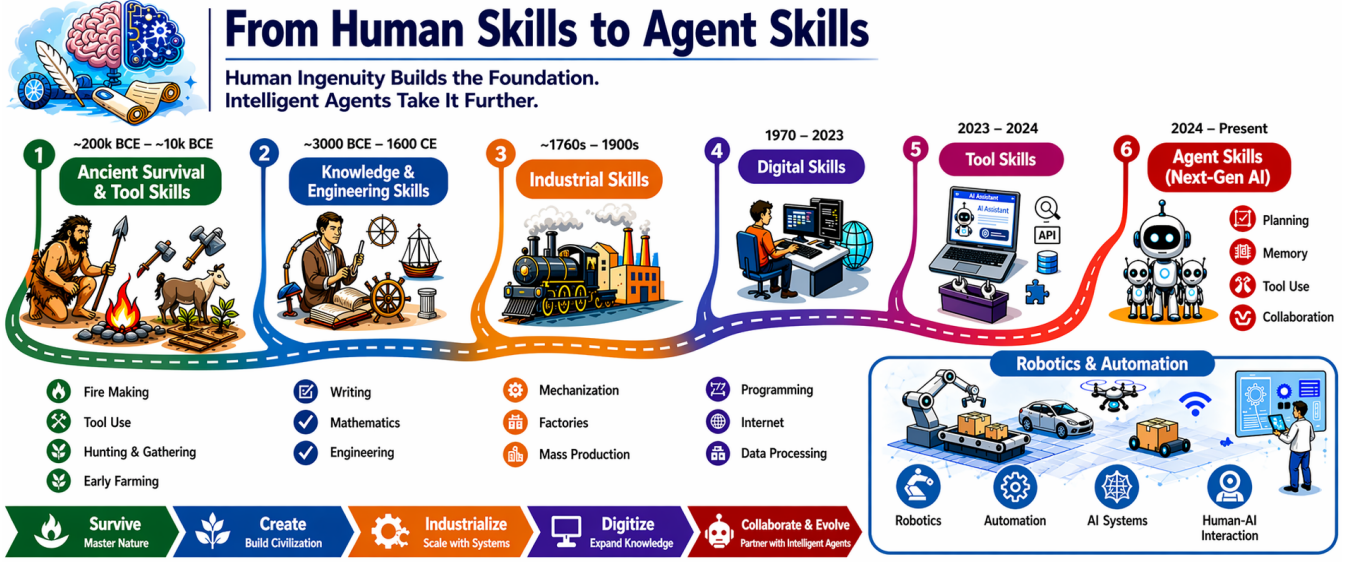


Fig. 1: Historical evolution of skills, from embodied human survival and craftsmanship to engineering, industrial, digital, and agent-era skill systems.

representative ecosystem resources, application settings, and synthesize open challenges in quality control, safety, cost, interoperability, maintenance, and long-term capability governance.

In summary, this survey makes the following contributions:

- We identify agent skills as a foundational component of modern LLM agent ecosystems, explicitly characterizing their relationship with agents and articulating their pivotal role in bridging the procedural gap between raw tool access and robust task execution.
- We organize existing research on agent skills around four core lifecycle stages —skill representation, skill acquisition, skill retrieval, and skill evolution — and review representative methods within each stage.
- We summarize representative agent skills platforms, application scenarios, and open challenges.
- We outline promising research directions for agent skills.

The remainder of this paper is organized as follows. Section II introduces core concepts and formal preliminaries. Section III presents the skill taxonomy used in this survey. Sections IV, V, and VI discuss skill acquisition, retrieval, and evolution, respectively. Section X discusses related work. Finally, Sections VIII and XI outline future directions and conclude the paper.

II. PRELIMINARIES

In this section, we introduce core preliminaries for skill-centric LLM agent ecosystems, including LLM-based agents, the procedural gap, the transition from tools to skills, and the lifecycle taxonomy used in this survey.

A. LLM-Based Agents

An LLM-based agent is a system that uses a language model as its core reasoning engine to autonomously complete

multi-step tasks through iterative interaction with an environment [4], [18], [20]. Its operation follows a perception–reasoning–action loop:

$$o_t \rightarrow r_t \rightarrow a_t \rightarrow o_{t+1}, \quad (1)$$

where o_t is the current observation, r_t is the reasoning state, and a_t is the resulting action. Recent systems such as Open-Claw and Manus exemplify this shift from passive response generation to action-oriented execution [7], [8]. What separates modern agents from standalone LLMs is that they can *act*: they query external systems, invoke tools, write and execute code, and coordinate with other agents [5], [6], [21]. That action capability is the foundation of the capability stack studied in this survey.

B. Agent Knowledge and the Procedural Gap

Agent behavior depends not only on the reasoning ability of the base model, but also on what knowledge is available when the agent must act. Here, we provide a simple distinction between passive and active knowledge to clarify where agent skills sit in the capability stack.

Passive knowledge is absorbed into the model’s parameters before deployment through pre-training, supervised fine-tuning, and alignment procedures such as RLHF [1]–[3]. It includes factual associations as well as diffuse procedural priors such as instruction following, decomposition, and code or plan generation. These priors support general reasoning, but they remain static, opaque, and often weak in specialized or rapidly changing domains.

Active knowledge is obtained at runtime through interaction with the environment [15], [21], [22]. It includes retrieving documents, invoking APIs and tools, accessing MCP servers, executing externalized agent skills, and observing outcomes. Active knowledge is more dynamic and grounded than parametric knowledge, but access alone does not determine *what*

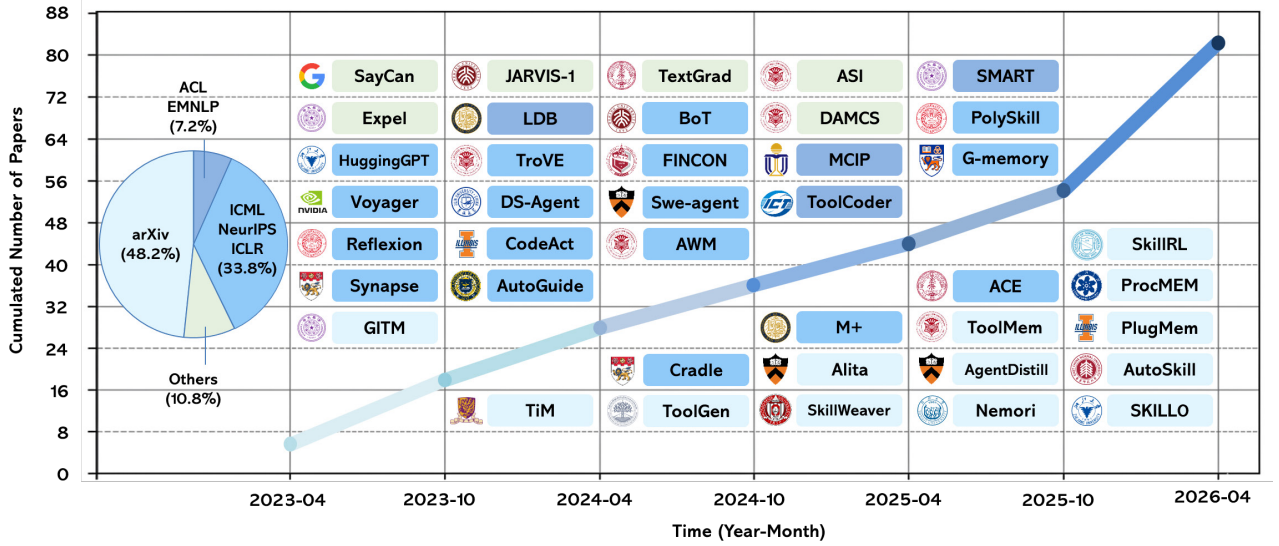


Fig. 2: Growth of research on agent skills from April 2023 to April 2026. The figure shows the cumulative number of representative papers together with example systems.

should be used, *when* it should be used, *how* it should be sequenced with other capabilities, or *how* its outputs should be validated.

To bridge this procedural gap, agent skills package procedural knowledge into reusable procedural artifacts that can be stored, retrieved, revised, and governed across tasks. The rest of this survey therefore focuses on this active procedural layer rather than on agent knowledge as a whole.

C. From Tool Skills to Agent Skills

The evolution of active LLM systems can be read as a transition from tool augmentation to skill augmentation. Tool-augmented agents made external capabilities practically usable at scale; skill-augmented agents emerged when researchers recognized that capability access alone was not enough, and that agents also needed reusable procedural artifacts for how those capabilities should be orchestrated in context.

Tools such as search engines, code interpreters, databases, and domain-specific APIs extend agents far beyond parametric memory alone [11], [21], [22], [103]. Standards such as the Model Context Protocol (MCP) further reduce integration friction by providing a unified mechanism for discovery and invocation across heterogeneous providers [10]. In effect, MCP makes it easier for an agent to *reach* external capabilities.

But access does not by itself yield reliable behavior. A tool exposes an *atomic capability*: it specifies *what* can be done, not *how* it should be used. A search tool does not say when search is preferable to memory retrieval; an API tool does not say what to do when the schema changes; a code interpreter does not say how outputs should be validated. MCP and similar infrastructures solve an *interoperability* problem, not the *procedural* problem of turning multiple tool calls into a robust workflow. As task complexity grows, that orchestration burden still falls back on the LLM at inference time and

becomes a major source of brittleness [4], [19]. This gap is addressed by introducing reusable procedural artifacts that specify when and how external capabilities should be applied.

D. Skills: Definition and Formalization

Rather than treating tool calls as isolated operations, systems store and reuse successful procedures as explicit *skills*.

Definition 1 (Agent Skill). A *skill* is a reusable procedural artifact with bounded scope that externalizes task-focused know-how: not only what can be done, but when to act, how to execute, what heuristics and failure modes matter, and how to judge completion [25], [27]. Formally, a skill is modeled as a tuple

$$S = (M, \mathcal{R}, \mathcal{C}),$$

where M is the root instruction document that the agent can load and follow; $\mathcal{R} = \{r_1, \dots, r_K\}$ is a set of auxiliary resources—reference documents, reusable templates, executable scripts, or domain artifacts that extend what the skill accomplishes beyond M alone [27]; and $\mathcal{C}$ encodes applicability conditions that govern when the skill should be retrieved and applied, expressed as metadata, natural-language descriptions, or embeddings [12], [13], [27]. The tuple need not be fully instantiated in every system, but captures how procedural knowledge becomes an explicit, inspectable object suitable for storage, retrieval, and orchestration.

Together, these components externalize task-focused know-how: not only *what* can be done, but *when* to act, *how* to execute, what heuristics and failure modes matter, and how to judge progress and completion. The tuple need not be fully instantiated in every system, but captures how procedural knowledge becomes an explicit, partially inspectable object suitable for storage, retrieval, and orchestration. In practice,

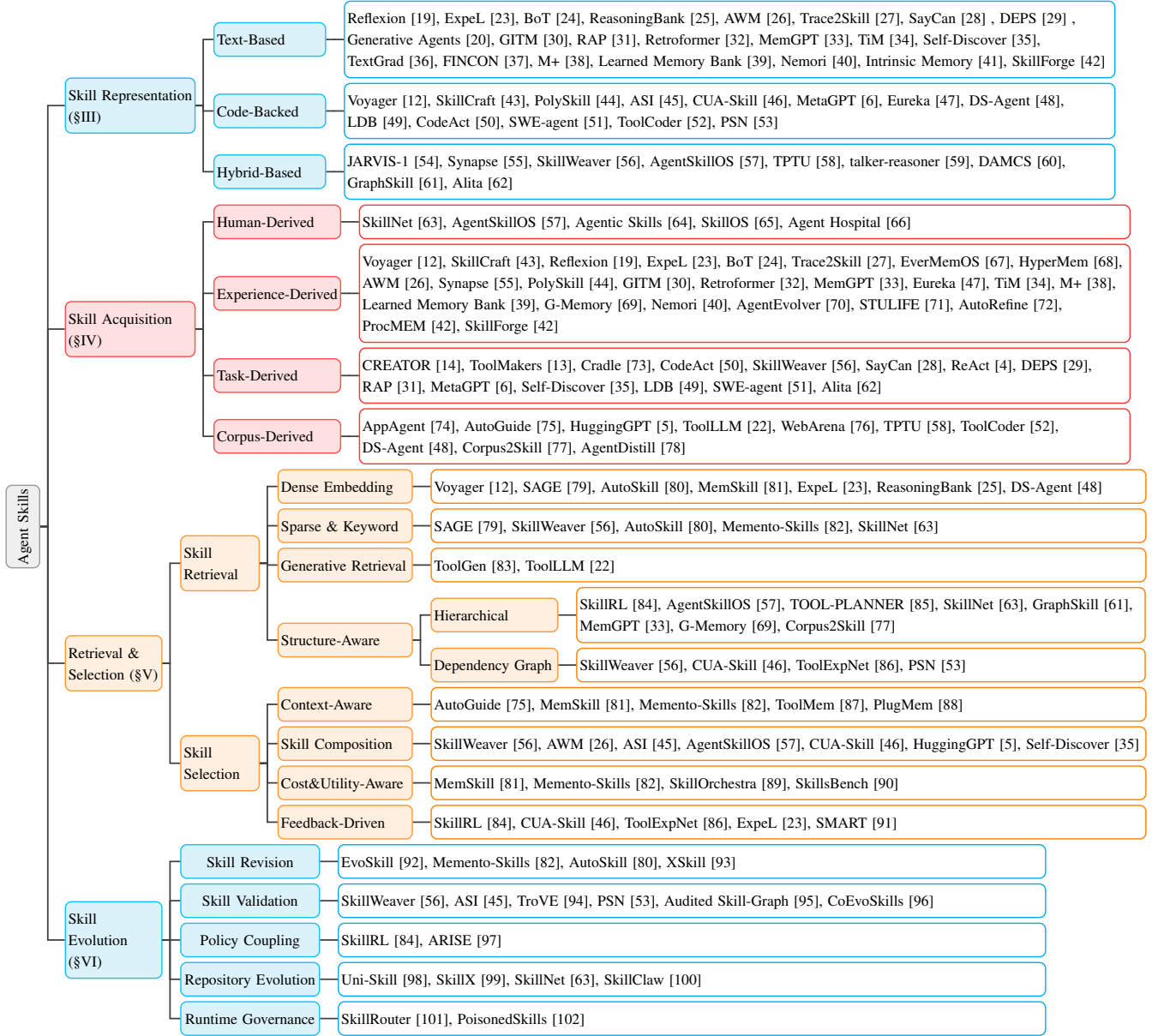


Fig. 3: The taxonomy for agent skills in this survey.

agent skills are increasingly managed through dedicated platforms such as SkillNet, ClawHub, SkillHub, SkillsMP, and Skills.sh (Table I).

Skills differ from raw tools and MCP servers in that they encode *situated* procedural knowledge (triggers, sequencing, fallbacks, pitfalls) and appear as *bounded, reusable artifacts* that can be loaded, inspected, shared, and revised without becoming an undifferentiated handbook—a constraint that matters as libraries scale [12], [13], [25], [27], [57], [104]. They need not be tool-centric: cognitive skills (e.g., review checklists, analysis workflows) mainly use internal knowledge but still supply structure and reuse beyond ad-hoc prompting. Tools expose operations; skills package know-how for using them in context.

E. The Ecosystem of Agent Skills

Agent systems should be understood as ecosystems rather than isolated reasoning modules. Skills may be created from

TABLE I: Representative agent-skill platforms.

Platform	Scale	Web
SkillNet	300k+	
ClawHub	40k+	
SkillHub	80k+	
SkillsMP	700k+	
Skills.sh	90k+	

demonstrations, traces, documents, or feedback; indexed in repositories; retrieved and selected under task, latency, or budget constraints; executed with tools, memory, and other agents; and later revised, validated, or retired as environments change. Because these stages are tightly coupled, errors can propagate across the lifecycle: incomplete skills are harder to retrieve, weak retrieval signals can activate unsuitable procedures, and stale dependencies can break otherwise useful behavior. In the following sections, we review each core lifecycle module

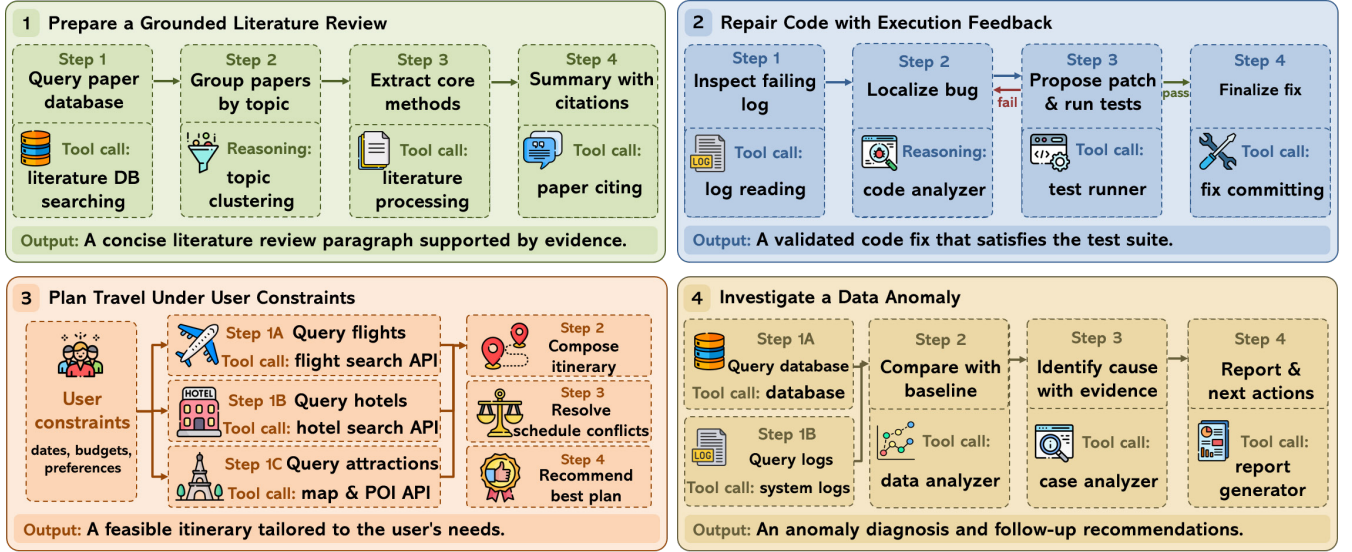


Fig. 4: Illustrative Examples of Agent Skills.

in this ecosystem. As illustrated in Fig. 3, our taxonomy organizes existing studies from this lifecycle perspective.

III. SKILL REPRESENTATION

Building on Section II, we classify how agent skills are *packaged* in practice. Each skill consists of an instruction-based main document M , optional auxiliary resources $\mathcal{R}$, and trigger conditions $\mathcal{C}$. Semantically, skills externalize procedural knowledge, including operational structure, branching heuristics, and normative constraints [105], with M serving as the primary human-readable representation. Although M may take different forms, such as brief reminders, checklists, standard operating procedures (SOPs), or more detailed workflows, these variations are secondary to our taxonomy. The more consequential distinction across systems lies in how $\mathcal{R}$ is configured.

In practice, however, M and $\mathcal{C}$ are often not sufficient on their own. Many skills rely on auxiliary resources $\mathcal{R}$ to provide domain references, reusable artifacts, or executable support, extending what the skill can accomplish beyond what is encoded in M alone.

A. Taxonomy by Resource Configuration

We classify skills into three configurations based on the type of resources in $\mathcal{R}$: text-based, code-based, and hybrid.

1) *Text-backed skills*: $\mathcal{R}$ consists of textual artifacts such as references, examples, templates, rubrics, or schemas. These resources improve grounding and reuse without introducing executable dependencies.

2) *Code-backed skills*: $\mathcal{R}$ consists of executable artifacts such as scripts, helper functions, notebooks, or wrappers [12], [13]. This enables repeatable subtasks and stronger operational determinism, but brings software packaging into the skill lifecycle: versioning, testing, and dependency management all become ongoing costs.

3) *Hybrid-resource skills*: $\mathcal{R}$ combines textual and executable artifacts [10], [27], aiming to preserve interpretability while supporting deterministic execution. The coordination burden is highest here, as consistency must be maintained across documents, code, and their bindings.

Example. Fig. 4 illustrates four representative examples of agent skills across different task scenarios. Each skill is organized as a reusable procedure composed of multiple steps, where each step may involve reasoning, tool invocation, or interaction with external resources. For example, a literature-review skill queries paper databases, groups papers by topic, extracts core methods, and produces a citation-supported summary. A code-repair skill inspects failing logs, localizes bugs, proposes patches, runs tests, and iteratively revises the fix based on execution feedback. Similarly, travel planning and anomaly investigation demonstrate how skills can coordinate parallel tool calls, satisfy user constraints, compare evidence, and generate actionable outputs.

B. Comparison and Summary

The three configurations differ not in the presence of M , but in what surrounds it. Text-based resources improve understanding; code-based resources improve execution reliability; hybrid configurations pursue both at the cost of greater complexity. The degenerate case—near-empty $\mathcal{R}$ —places the full burden on M alone. At ecosystem scale, this taxonomy shapes how skills are indexed, validated, maintained, and orchestrated [57], and provides a unified account of lightweight reminders, workflow packages, and richer skills with attached code and references.

IV. SKILL ACQUISITION

Skill acquisition is the process of constructing or generating new skills. In practice, this means obtaining reusable procedural guidance or executable skill artifacts.

At a broad level, the literature can be organized into four families according to the dominant direct source from

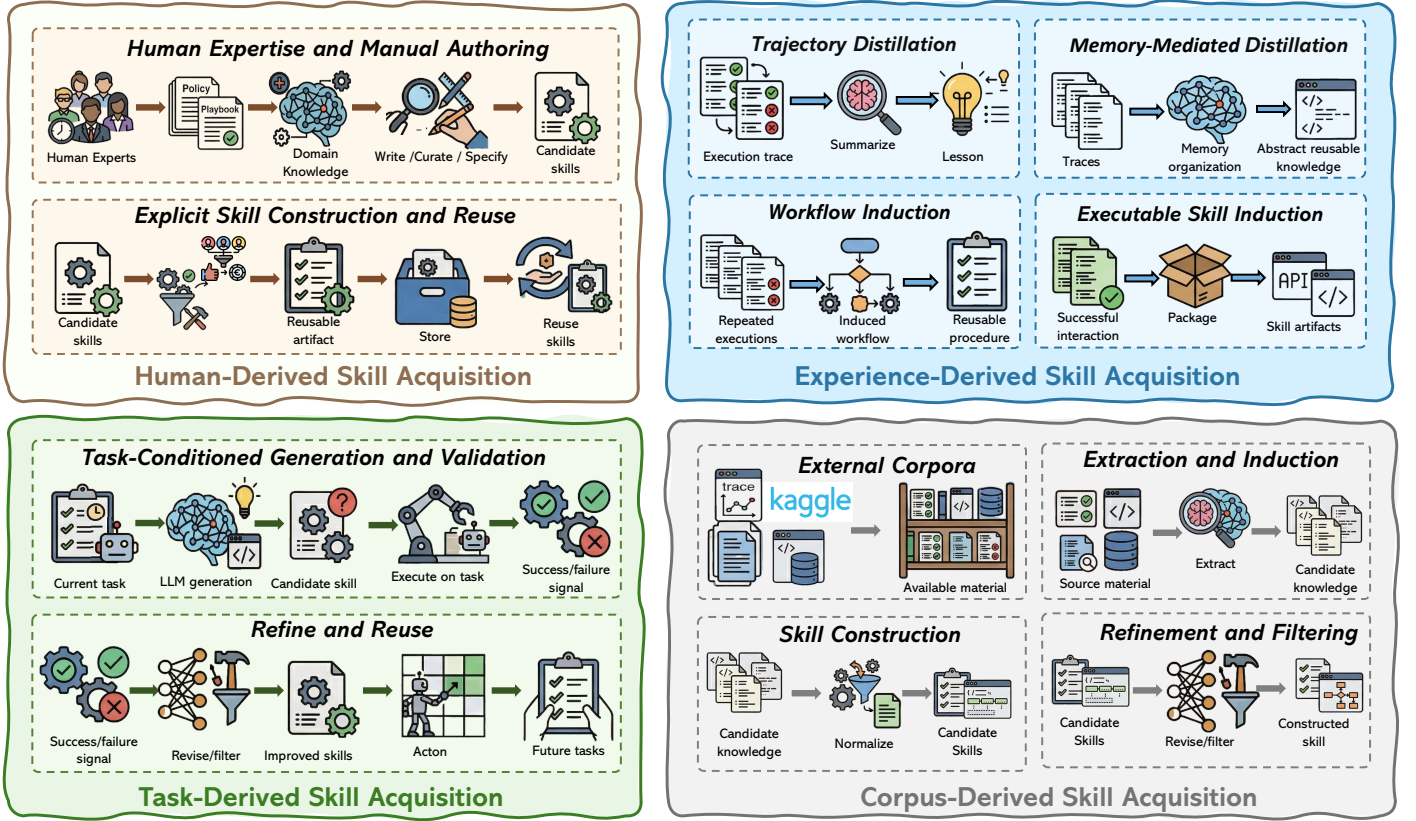


Fig. 5: Overview of skill acquisition methods.

which a skill is obtained: ❶ human-derived acquisition, ❷ experience-derived acquisition, ❸ task-derived acquisition, and ❹ corpus-derived acquisition. Human-derived acquisition obtains skills directly from expert knowledge and manual curation. Experience-derived acquisition builds them from trajectories, exemplars, or past executions. Task-derived acquisition constructs them on demand from the requirements of the current task. Corpus-derived acquisition extracts skills from external textual or structured corpora such as documents, repositories, and interface traces. Fig. 5 summarizes this broader landscape before the more focused discussion below.

A. Human-Derived Acquisition

Human-derived acquisition obtains skills directly from domain experts. In this group, people explicitly write reusable procedures, define their intended scope, and attach supporting materials or usage constraints when needed.

Through repeated practice, they accumulate tacit know-how, exception-handling strategies, and context-sensitive judgment, which can then be externalized as reusable skills. For example, doctors may summarize diagnostic experience into treatment procedures, medication prescriptions, and drug-specific usage recipes; engineers may codify troubleshooting workflows and operational playbooks; and policy experts may formalize review criteria and safety constraints. By doing so, human-derived acquisition therefore converts domain knowledge and professional experience into explicit procedural artifacts that agents can inspect, reuse, and adapt.

Discussions. The trade-offs of human-authored acquisition are straightforward. Its strength is precision: experts can encode tacit judgment, conventions, and safety-critical rules with fine semantic control. Its weakness is scalability, since manual curation is slow to grow and maintain; in practice, human-derived skills often serve as a seed layer for more automated acquisition. Fig. 6 shows the recent growth and diversification of skills, based on statistics from SkillsMP (<https://skillsmp.com/>). This trend reflects a gradual shift: more skills designed by domain experts are being incorporated into agent platforms, as summarized in Table I. Such expansion greatly improves agents’ ability to perform specialized, domain-specific tasks.

B. Experience-Derived Acquisition

Experience-derived acquisition treats an agent’s past runs—execution traces, exemplars, interaction histories, and feedback—as raw material from which it abstracts recurring patterns (including failures) into reusable, transferable skills, rather than leaving that information only as transient memory. This mirrors a familiar learning intuition—mine experience, distill procedure—and is currently the most heavily studied acquisition family, with the widest spread of concrete mechanisms and systems.

What chiefly differentiates methods is not the provenance of experience—typically traces, logs, and feedback from prior runs—but how that material is processed before it hardens into a skill. Concretely, systems may select and retain successful

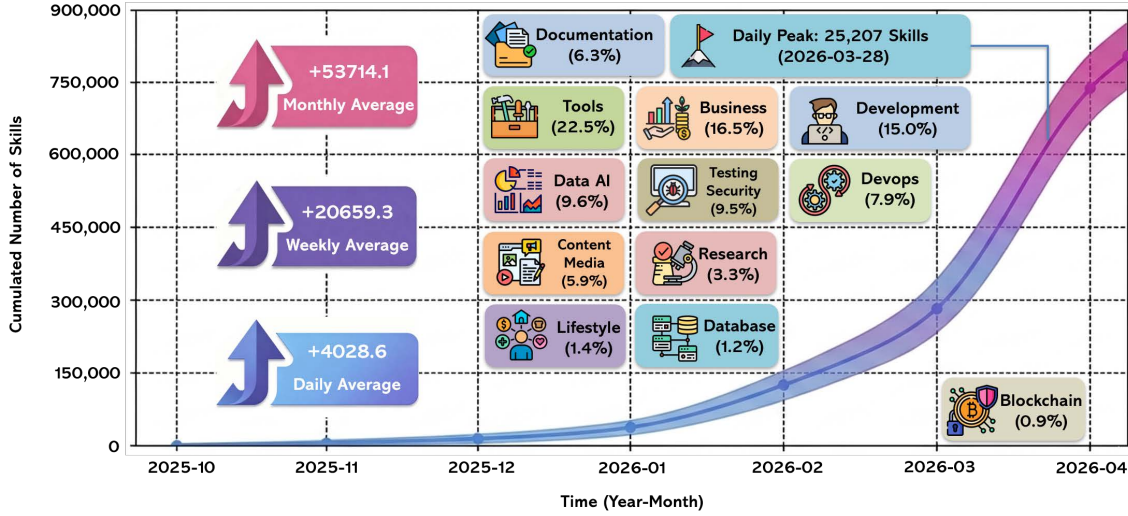


Fig. 6: The trend of cumulated number of human-derived skills over time.

episodes as reusable exemplars; abstract long trajectories into compact lessons, heuristics, or declarative guidance; organize scattered evidence into structured memory that transfers across tasks; or package distilled know-how into richer artifacts such as workflows, APIs, or executable modules. Viewed this way, experience-derived acquisition is a pipeline from execution traces to skill artifacts, mediated by experience-processing operations (selection, abstraction/summarization, memory organization, and procedural packaging). These steps are seldom isolated: a single pipeline often chains several of them. The remainder of this section walks through each operation in turn.

A first common operation is **selection**. Since not all experience is equally useful for skill construction, systems often begin by filtering historical trajectories and retaining only those that are successful, informative, or representative. Voyager is a canonical example: it keeps successful embodied trajectories and uses them as the basis for later executable skill construction [12]. SkillCraft follows a similar pattern in tool-use settings, where useful tool trajectories are selected before being further abstracted into executable skills [43]. In this sense, selection acts as an experience-quality control stage: it determines which parts of execution history become reusable raw material, while failed or noisy traces are either discarded or passed to abstraction-oriented methods as corrective evidence.

A second operation is **summarization and abstraction**. Here, concrete traces are compressed into reusable procedural knowledge such as lessons, heuristics, guidelines, or declarative skill descriptions. Reflexion extracts verbal reflections from failed attempts, turning specific errors into short corrective rules for future use [19]. ExpeL goes beyond single-trajectory reflection by abstracting higher-level lessons from accumulated successes and failures [23]. FINCON stores manager-level insights distilled from prior financial decision episodes as reusable guidance, while Buffer of Thoughts distills prior problem-solving experience into reusable reasoning templates that can later be instantiated on new tasks [24], [37]. Think-in-Memory (TIM) likewise uses post-thinking to

extract reusable natural-language experience, and Trace2Skill pushes this direction further by hierarchically consolidating local trajectory-level lessons into explicit reusable skill artifacts [27], [34]. Across these methods, the shared pattern is that episodic executions are not preserved in full detail, but compressed into more compact knowledge units that are easier to retrieve and reuse.

A third operation is **memory organization**. Instead of treating each trajectory independently, some methods reorganize accumulated experience into more persistent and structured memory forms. Think-in-Memory integrates reflection into memory updates so that new executions reshape later reasoning rather than remaining as isolated records [34]. G-Memory makes this structuring more explicit by organizing collective experience into hierarchical memory graphs that support reuse at different abstraction levels [69]. Nemori provides a related example in which episodic interactions are further distilled into more stable semantic memory, while Intrinsic Memory Agents maintain structured contextual memory so that accumulated experience can remain usable over longer horizons [40], [41]. AgentEvolver summarizes cross-task experience into reusable natural-language guidance units for later exploration, and DAMCS organizes multimodal past interactions into a hierarchical knowledge-graph memory that supports retrieval and structured communication for cooperative planning [60], [70]. These systems differ in representation and scope, but all illustrate the same design move: past execution is reorganized into structured memory that remains available for later decision making and adaptation.

A fourth operation is **procedural induction and packaging**. Rather than extracting only textual guidance, some systems convert repeated successful executions into richer reusable procedures such as workflows, plans, APIs, or executable modules. Agent Workflow Memory is a representative case because it induces reusable workflows directly from prior interaction traces and stores them as workflow skills with triggers, multi-step procedures, and parameter slots [26]. Related systems such as JARVIS-1, Synapse, and Ghost in the

TABLE II: Representative experience-derived methods organized by experience processing operators.

Method	Source	Form	Key Characteristics	Venue/Year
<i>Selection</i>				
Voyager [12]	Successful embodied trajectories	Code	Retains successful executable traces	NeurIPS’23
SkillCraft [43]	Successful tool trajectories	Code	Filters useful tool-use traces	arXiv’26
<i>Abstraction</i>				
Reflexion [19]	Failed trajectories	Text	Converts failures into lessons	NeurIPS’23
ExpeL [23]	Successes and failures	Text	Abstracts reusable lessons	AAAI’24
BoT [24]	Problem-solving traces	Text	Distills reasoning templates	NeurIPS’24
Trace2Skill [27]	Trajectories	Text	Consolidates trace-level skills	arXiv’26
FINCON [37]	Decision episodes	Text	Distills decision insights	NeurIPS’24
AgentEvolver [70]	Processed trajectory sets	Text	Summarizes cross-task experience	arXiv’25
<i>Memory Organization</i>				
TiM [34]	Interaction history	Text	Updates reflective memory	arXiv’23
G-Memory [69]	Collective experience	Text	Builds hierarchical memory	NeurIPS’25
Nemori [40]	Dialogue history	Text	Consolidates semantic memory	arXiv’25
Intrinsic Memory [41]	Interaction history	Text	Maintains contextual memory	arXiv’25
DAMCS [60]	Multimodal interactions	Hybrid	Organizes multimodal KG memory	arXiv’25
<i>Procedural Packaging</i>				
JARVIS-1 [54]	Successful multimodal task executions	Hybrid	Retains task-plan procedures	TPAMI’25
Synapse [55]	Successful computer-control trajectories	Hybrid	Packages trajectory exemplars	ICLR’24
AWM [26]	Interaction traces	Text	Induces workflow skills	ICML’25
PolySkill [44]	Successful interaction traces	Code	Builds programmatic skills	ICLR’26

Minecraft likewise preserve more structured procedural support from successful experience, often retaining ordering, decomposition, or action dependencies [30], [54], [55]. Voyager, SkillCraft, and Trace2Skill move further toward reusable skill packaging, while executable-skill systems such as PolySkill show how successful interaction experience can be abstracted into directly callable programmatic skills rather than free-form lessons alone [12], [27], [43], [44]. The distinction of this operation is therefore not simply that it preserves more information, but that it packages experience into artifacts that are closer to direct reuse, whether as workflow templates, structured procedures, or executable modules.

Taken together, these methods show that experience-derived acquisition is not defined by a single output form. Its unifying feature is that skills are constructed from past execution, while the main design variation lies in how those traces are processed before reuse. Different systems therefore occupy different points along a spectrum from selected traces, to summarized lessons, to structured memory, to packaged workflows and executable skill artifacts, without implying that any one form is universally superior to the others.

C. Task-Derived Acquisition

Task-derived acquisition constructs skills directly from the requirements of the current task. The task itself acts as the trigger for generation: an LLM or a related synthesis module proposes a candidate workflow, script, tool wrapper, or other skill artifact, and execution outcomes determine whether it is discarded, revised, or retained. The key idea is therefore not

simply model generation, but task-conditioned skill construction followed by post-hoc retention or refinement.

For instance, CREATOR generates callable tools directly from task needs, while ToolMakers explicitly separates skill creation from skill use so that generated skills can later be reused rather than remaining tied to a single episode [13], [14]. Cradle and CodeAct synthesize procedural artifacts for immediate control and action, showing how a task can directly induce reusable operational procedures instead of only producing a final answer [50], [73]. Systems such as Alita and TROVE push this pattern further by coupling task-conditioned generation with execution-time validation and later retention, whereas SkillWeaver explicitly discovers API-like skills from web interaction, filters them through subsequent use, and refines them into reusable callable artifacts over time [56].

D. Corpus-Derived Acquisition

Corpus-derived acquisition derives skills from external textual or structured resources rather than from the current task or the agent’s own experience. Typical sources include documentation, software repositories, datasets, interface traces, and knowledge bases, and the core process is to distill these resources into reusable procedural skills.

Representative systems illustrate different instantiations of this process. AppAgent extracts procedural signals from interface structures, AutoGuide derives context-aware guidelines from external knowledge sources, and HuggingGPT and ToolBench/ToolLLM compile procedural guidance from model cards and API descriptions. In data-science settings, DS-

Agent follows a more case-driven route: it mines external competition resources (for example, Kaggle write-ups from gold- and silver-medal solutions across different types of tasks), abstracts recurring solution patterns, and turns them into reusable procedural guidance for subsequent tasks [5], [22], [74], [75].

E. Discussion

No single acquisition family dominates, and their practical importance is becoming more complementary rather than more exclusive. With the assistance of LLMs, human experts can now externalize domain knowledge into reusable skills much more easily than before, by drafting procedures, refining usage conditions, and packaging supporting resources with far lower authoring overhead. Similarly, corpus-derived acquisition has become increasingly effective because LLMs can transform documentation, repositories, case reports, and other external artifacts into more explicit procedural guidance, thereby expanding the accessible supply of candidate skills beyond what agents have personally experienced.

At the same time, the other two families remain equally important. Experience-derived acquisition is still the richest and most widely explored route, because execution traces provide a natural substrate for grounding skills in observed behavior and continuously improving them through filtering, abstraction, memory organization, and procedural packaging. Task-derived acquisition, in turn, is indispensable when agents face novel requirements that cannot wait for experts, corpora, or long-term experience accumulation, since it enables on-demand construction of candidate skills followed by validation and retention.

Taken together, these four families should not be viewed as competing alternatives, but as complementary routes for building robust skill ecosystems. Human-derived methods contribute semantic precision and high-trust expertise; experience-derived methods contribute behavioral grounding and diversity; task-derived methods contribute responsiveness; and corpus-derived methods contribute scalable cold-start coverage. In practice, the most capable skill libraries are likely to emerge from their combination, with LLMs serving as the common catalyst that lowers the cost of skill creation, transformation, and maintenance across all four routes.

V. SKILL RETRIEVAL

As agent systems accumulate reusable experience over time, the bottleneck gradually shifts from skill acquisition to skill access: the central question is no longer whether a useful skill exists, but whether the system can surface and activate the right skill at the right moment [12], [23]. This challenge becomes especially pronounced when the skill repository grows in both scale and heterogeneity, spanning natural-language instructions, workflow memories, executable APIs, and packaged skill artifacts with metadata and auxiliary resources [23], [26], [56], [63]. In such settings, skill use is better viewed as a multi-stage pipeline, where the system must first narrow the search space and then make an execution decision under the current task and environment state [26], [84]. This pipeline is

often richer than one-shot semantic matching: a runtime may first discover candidate skills through names, descriptions, or metadata; progressively disclose more detailed instructions and resources only when needed; and finally bind the selected skill to concrete tools, APIs, files, or subagents for execution [105].

This perspective motivates a distinction between *skill retrieval* and *skill selection*. *Skill retrieval* concerns how a large skill pool is reduced to a manageable candidate set, for example, through semantic matching, lexical lookup, generative access, or structure-aware search. *Skill selection*, in contrast, concerns which candidate skill should ultimately be invoked, whether multiple skills should be composed, and how such choices should adapt to the current observation, subgoal, and resource budget. Put differently, retrieval addresses *candidate recall*, while selection addresses *execution-oriented decision making*. Distinguishing the two stages provides a clearer analytical framework for this section, even though practical systems often interleave them tightly.

This distinction also clarifies why retrieving skills cannot be treated as a straightforward instance of conventional document retrieval. Unlike documents, skills are executable units: invoking them may trigger tool calls, workflow transitions, external side effects, and non-trivial costs [2,3,4]. Consequently, semantic relatedness alone is rarely sufficient. A useful skill must also be executable under the current state, satisfy relevant preconditions, interact properly with other skills, and deliver sufficient utility relative to its cost [26], [56], [84]. The remainder of this section therefore examines skill use as a two-stage problem: we first review how existing systems retrieve candidate skills, and then analyze how they rank, compose, and select them for execution.

A. Skill Retrieval

This subsection focuses on the first stage of the skill-use pipeline: how a system retrieves a small set of candidate skills from a large repository for downstream decision making. Unlike the introduction, the emphasis here is not on why retrieval matters in general, but on how candidate recall is operationalized in existing systems. We organize prior work by the primary signal used to retrieve skills, including dense semantic matching, sparse lexical matching, direct generation of skill identifiers, and retrieval guided by structural relations among skills.

1) *Dense Embedding Retrieval*: Dense retrieval is the most common strategy when tasks are described in natural language and skills are paired with textual descriptions, workflow summaries, or other semantic metadata. Its basic idea is to map the current task and candidate skills into a shared embedding space, and then retrieve the most relevant skills by vector similarity. Voyager [12] remains the canonical example because semantic retrieval is applied to textual skill descriptions even though the stored skills themselves are executable code. Recent skill-centric systems such as SAGE [79], AutoSkill [80], and MemSkill [81] show the same pattern across programmatic skills, structured skill records, and explicit skills for memory management. Lighter-weight memory-oriented variants such as ExpeL [23] and ReasoningBank [25] extend this line by

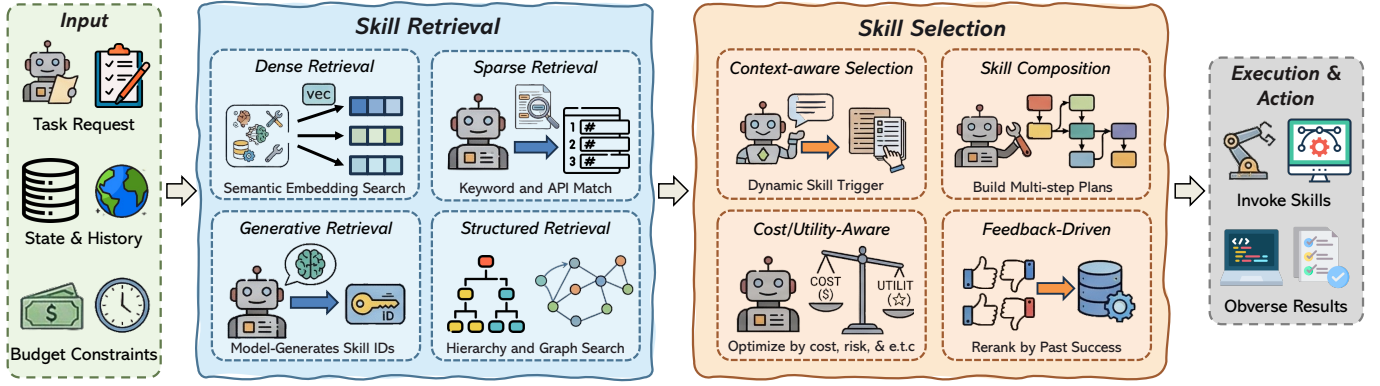


Fig. 7: Skill retrieval and selection.

applying dense retrieval to experiential lessons or structured reasoning memories rather than fully packaged executable skills.

This makes dense retrieval the natural entry point when task formulations vary widely but the system still needs to reach reusable skills through a shared semantic layer. The same flexibility also explains why dense retrieval is rarely the whole story. Nearest neighbors in meaning are not always nearest neighbors in applicability, especially once libraries expose richer constraints than text similarity alone can capture [79], [81]. Accordingly, dense retrieval often opens the candidate set, while later stages refine it with metadata, structure, or execution-aware checks.

2) *Sparse and Keyword Retrieval*: Sparse retrieval treats candidate recall as matching over explicit symbolic fields and metadata attached to skill artifacts. In the current skill literature, it operates primarily through lexical or field-aware access to those explicit descriptors. SAGE [79] illustrates this through its “Query N-gram” retrieval variant, while SkillWeaver [56] relies heavily on explicit interface descriptions and applicability-related metadata. AutoSkill [80], Memento-Skills [82], and SkillNet [63] further show that once skills are externalized as structured artifacts, lexical matching and metadata search become useful complements to semantic retrieval rather than outdated baselines.

Compared with dense retrieval, this line is narrower but often more trustworthy when the library exposes stable names, interface fields, or trigger cues. It is therefore most useful in settings where retrieval errors come from symbolic mismatch rather than from missing semantic coverage. Once the request becomes paraphrastic or underspecified, however, lexical evidence degrades quickly, which is why sparse retrieval usually sharpens or filters a broader candidate pool instead of replacing it [63], [80], [82].

3) *Generative Retrieval*: Generative retrieval treats candidate recall as identifier generation, so the model produces the target tool or skill token directly during decoding instead of consulting a separate retrieval index. This idea is most explicit in ToolGen [83], which reformulates candidate access as identifier generation during decoding. ToolLLM [22] provides a related but looser example in which generation and large-scale tool calling are tightly coupled, even though the retrieval

stage is not as explicitly reformulated.

Conceptually, this line is attractive because it removes the boundary between candidate recall and downstream action generation. That tight coupling, however, also makes it harder to separate retrieval quality from calling behavior, and harder to guarantee coverage or identifier validity in large candidate spaces. Within a skill survey, generative retrieval is therefore better treated as a neighboring mechanism than as the dominant pattern for explicit skill libraries.

4) *Structure-Aware Retrieval*: Structure-aware retrieval assumes that skill libraries have internal organization that should guide recall rather than treating all candidates as a flat pool. Two recurring forms are most visible in the current literature: *hierarchical retrieval*, which narrows the search space progressively, and *dependency-aware retrieval*, which filters candidates through structural compatibility constraints.

- *Hierarchical retrieval* organizes candidates through coarse-to-fine structure, so retrieval first reduces the search space at a higher level and only then resolves finer-grained choices. SkillIRL [84] and AgentSkillOS [57] are the clearest representatives because both use an explicit hierarchy to move from broad skill regions toward more specific candidates, while TOOL-PLANNER [85] and SkillNet [63] show the same intuition in toolkit- or ecosystem-level settings. GraphSkill [61] provides a related example by traversing structured documentation from high-level categories to leaf-level algorithmic entries, so coarse pruning happens before finer-grained retrieval. This design is especially useful when the main difficulty is search-space reduction over large repositories rather than checking whether a retrieved skill is currently executable.

- *Dependency-aware retrieval*, by contrast, focuses less on narrowing the space and more on ruling out candidates that violate prerequisites, state constraints, or composition requirements. SkillWeaver [56] illustrates this through prerequisite and precondition filtering, CUA-Skill [46] makes structural compatibility part of skill use, and ToolExpNet [86] shows how dependency relations can directly reshape which candidates remain plausible after retrieval. This view becomes important once the question is not only which skill is relevant, but also which skill can participate in a valid execution path under the current conditions.

5) *discussion*: Taken together, these paradigms suggest that skill retrieval is not a single matching problem, but a trade-off among semantic flexibility, symbolic precision, and structural executability. Dense retrieval is most effective when tasks are expressed in open-ended natural language, while sparse retrieval is more reliable when success depends on exact names, parameters, or interface-level cues. Generative retrieval goes one step further by folding retrieval into decoding itself, but this comes with weaker control over coverage and identifier validity. Structure-aware retrieval, in turn, is most useful when skills form a compositional space with hierarchies or dependency graphs that flat similarity cannot capture. Overall, the field is moving away from one-shot relevance matching toward multi-signal, execution-aware candidate recall.

B. Skill Selection

Following the retrieval stage, this subsection examines how existing systems make selection decisions over the retrieved candidate set. We organize prior work into four common perspectives: context-aware dynamic selection, skill composition, cost- and utility-aware selection, and feedback-driven re-ranking. These categories are better understood as complementary decision dimensions than as strictly mutually exclusive classes. A practical system may therefore combine multiple perspectives at once, such as dynamic selection with cost-aware routing or compositional planning with feedback-based updating.

1) *Context-aware dynamic selection*: Context-aware dynamic selection treats skill choice as an online decision process conditioned on the current observation, subgoal, and interaction history. Rather than selecting a skill once from a fixed candidate set, this perspective revises the choice as execution unfolds. This pattern is most visible when the same skill bank must support changing local states rather than only static task descriptions. AutoGuide [75] illustrates this clearly by selecting context-conditional guidelines from the current environment state. MemSkill [81] and Memento-Skills [82] extend the same idea to evolving skill banks, where routing depends on the current state together with changing internal context.

2) *Skill Composition*: Skill composition views skill selection as the problem of choosing and organizing multiple skills rather than selecting a single best candidate. Under this view, complex tasks are solved by assembling a sequence, set, or workflow of reusable skills. The central question is therefore not only which skills are relevant, but also how they should be ordered and connected for execution. SkillWeaver [56], AWM [26], and ASI [45] show this most clearly by treating APIs, workflows, or programmatic routines as reusable units that can be composed into larger behaviors. AgentSkillOS [57] and CUA-Skill [46] push this one step further by making orchestration structure explicit through formal dependency patterns. HuggingGPT [5] remains a useful early reference because it established the workflow view of selection, even though it is more tool-centric than later skill-oriented systems.

This perspective anchors a large part of the current skill-selection literature because many tasks are not solved by

choosing one best skill. Instead, the system must decide how reusable modules are sequenced, grouped, or nested into a larger executable behavior. That expressiveness comes with new failure modes, since interface compatibility, ordering constraints, and error propagation all become part of the selection problem [46], [56], [57]. Accordingly, skill composition is usually coupled with planning or structural guidance rather than treated as simple top-k ranking.

3) *Cost & Utility-aware selection*: The core idea of these methods is that systems should not simply prefer the most relevant skill, but should instead consider expected benefit together with cost, risk, or side effects. MemSkill [81] and Memento-Skills [82] show the method side of this shift, because their routing policies are shaped by downstream utility or behavioral success rather than only surface relevance. SkillOrchestra [89] provides a clearer cost-aware variant that routes among candidate agents based on current skill demand, expected competence, and deployment cost. SkillsBench [90] provides the clearest empirical motivation by showing that even curated skills can have negative utility on some tasks.

This perspective matters because deployed systems pay for wrong selections not only in accuracy, but also in wasted computation, latency, and unnecessary execution. At the same time, the literature still uses heterogeneous notions of utility, without a shared formal objective for skill selection. So unlike the other three subclasses, cost- and utility-aware selection is best read as an emerging design criterion that should inform selection, not yet as a mature standalone family of methods.

4) *Feedback-driven reranking*: Feedback-driven re-ranking updates skill preferences using historical execution signals rather than relying only on the current candidate-query match. The key idea is that success or failure history can be used to reorder candidates and improve later decisions. SkillRL [84] and CUA-Skill [46] are the strongest skill-centric examples because execution outcomes directly alter later priorities over reusable skills. ToolExpNet [86], ExpEL [23], and SMART [91] provide adjacent variants in which experience modifies later preference ordering through feedback-informed guidance, memory revision, or calibration.

Unlike utility-aware selection, which changes what the system is trying to optimize, feedback-driven re-ranking changes how past outcomes rewrite later preference orderings. This mechanism is most valuable for long-running agents, where today's mistake should become tomorrow's ranking signal. But feedback is rarely clean, and in current systems it is often entangled with memory editing or policy adaptation rather than an isolated reranker [84], [86]. For that reason, feedback-driven re-ranking usually appears as an augmentation layer over a broader selection pipeline.

5) *discussion*: Taken together, these paradigms show that skill selection is better understood as a policy problem than as a final ranking step over retrieved candidates. Context-aware methods emphasize adaptation to the current state. Compositional methods emphasize coordination across multiple skills. Cost-aware methods emphasize explicit trade-offs between utility and budget. Feedback-driven methods emphasize learning from execution outcomes over time. These dimensions are not mutually exclusive, and many practical systems combine

several of them at once. Overall, the literature suggests that skill selection is moving from static relevance choice toward sequential, execution-aware decision making.

C. Design Dimensions in Retrieval and Selection

In this subsection, we highlight the main design dimensions that shape both stages in terms of skills retrieval and selection.

1) *Representation from the Retrieval/Selection Perspective*: Skill representation matters because retrieval and selection can only use the signals that the representation exposes. Pure text skills mainly expose semantic and lexical cues, whereas pure code skills are often hard to retrieve or rank unless additional names, signatures, docstrings, or textual summaries are available [12]. Hybrid skills expose both semantic descriptions and execution-relevant structure, which makes them easier to retrieve and easier to constrain during downstream selection [56], [63]. From this perspective, representation is best understood as what the system can see and act on during retrieval and selection.

2) *State and Applicability*: State and applicability form the main bridge between retrieval and selection. A skill may be relevant to the task description but still be unusable under the current observation, environment state, or prerequisite constraints [56], [84]. State determines which candidates survive precondition or dependency checks during retrieval, and whether previously retrieved candidates remain appropriate after intermediate observations or failures during selection.

3) *Granularity and Composition*: Granularity determines what the system is actually retrieving or selecting. Some systems operate at the level of single primitive skills, while others retrieve workflow memories, executable modules, or groups of composable skills [26], [45]. As granularity increases, the problem shifts from choosing one candidate to organizing a larger executable structure. The granularity of the retrieved object therefore directly shapes whether downstream selection is a routing problem or an assembly problem.

4) *Objectives, Feedback, and Evaluation*: Objectives, feedback, and evaluation should be analyzed together because they all determine what the system is optimizing for and how that objective is updated over time. Selection is not only about relevance, but also about whether a skill is worth invoking under constraints such as utility, reliability, or execution burden [81], [82], [90]. Execution feedback then changes future behavior by updating which candidates are preferred, suppressed, or refined [82], [84]. This makes evaluation especially difficult, because standard retrieval metrics such as top-k recall do not measure whether the final execution succeeded or whether the chosen skill produced positive net utility [90]. SRA-Bench makes this gap concrete by separately examining skill retrieval, skill incorporation, and final task execution [106]. However, a complete evaluation framework still needs to connect candidate quality, execution outcome, cost efficiency, and feedback-driven adaptation rather than measuring each in isolation.

VI. SKILL EVOLUTION

Human skills improve through correction, consolidation, and reuse rather than accumulation alone. A chess player

improves when recurring mistakes reshape later pattern recognition; a craftsperson improves when failures revise an executable routine; and a clinician improves when prior cases update an earlier diagnostic heuristic. The common mechanism is *progressive refinement*: feedback changes the reusable procedure behind a capability rather than merely extending a record of experience.



Fig. 8: From human skill refinement to agent skill evolution.

For agent systems, this analogy is useful only after skill evolution is separated from skill acquisition. Acquisition explains how a candidate skill is first obtained. Evolution asks how an already formed skill artifact is revised, validated, optimized, shared, and governed after it has been formed. Fig. 8 provides the motivating analogy, while Fig. 9 gives the artifact-level refinement process used in the rest of this section. The evolving object may be a `Skill.md` file, a skill folder, a program API, a toolbox function, a skill bank, or a skill graph. Trajectories, task outcomes, user feedback, rewards, failures, and skill gaps are therefore update signals for the skill substrate, not the organizing categories of this section.

Skill revision changes the content of a stored artifact. Skill validation tests whether a proposed change should survive. Policy coupling lets the validated skill and the controller that uses it adapt together. Repository evolution turns accepted updates into an indexed and synchronized skill substrate. Run-time governance then retrieves candidates from that substrate

TABLE III: Representative methods for skill retrieval. The table compares each method by subclass, core design, structural prior or selection tags, decision inputs, and publication venue.

SubClass	Method	Core Design	Tags	Decision Inputs	Venue/Year
<i>Skill Retrieval</i>					
Dense	Voyager [12]	Text-described code skill retrieval	Flat	task	TMLR'24
	SAGE [79]	Program skill similarity retrieval	Flat	task instruction	arXiv'25
Sparse / keyword	AutoSkill [80]	Hybrid rewritten-query retrieval	Flat	rewrite + metadata	arXiv'26
	Memento-Skills [82]	Behavior-routed hybrid retrieval	Flat	state + query	arXiv'26
Generative	ToolGen [83]	Directly generate tool/skill identifier	Flat	task query	ICLR'25
Structure-aware	SkillIRL [84]	Hierarchical general-specific retrieval	Hierarchy	instruction+skill bank	arXiv'26
	SkillWeaver [56]	Precondition-aware API filtering	graph	state + metadata	arXiv'25
<i>Skill Selection</i>					
Context-aware	AutoGuide [75]	Trajectory-derived guideline routing	Ctx	context + trajectory	NeurIPS'24
skill composition	AWM [26]	Workflow memory reuse	Comp	task/subgoal	ICML'25
	ASI [45]	Verified program skill composition	Comp	task + prior skills	COLM'25
Cost/Utility	MemSkill [81]	Reward-coupled memory routing	Ctx+Util+Fb	span + memories	arXiv'26
Feedback reranking	CUA-Skill [46]	UI-aware failure reranking	Ctx+Comp+Fb	UI + memory + failures	arXiv'26
	ToolExpNet [86]	Graph-guided experience reranking	Ctx+Comp+Fb	graph + experience	ACLF'25

Note: In the **Tags** column, *Flat*, *hierarchy*, and *dependency graph* denote retrieval-side structural priors; *Ctx* = context-aware, *Comp* = composition, *Util* = utility-aware, and *Fb* = feedback-driven denote selection-side tags.

under later task conditions, routes them to execution, applies trust checks, and retires unsafe or stale skills.

The cycle closes only after governed reuse. Once a governed skill is executed, its outcomes can produce the next reward, failure, feedback, skill gap, or trust signal, which returns to the revision stage. Table IV compares artifact-changing methods that instantiate revision, validation, policy coupling, and repository evolution; runtime governance is discussed in prose because it controls reuse rather than directly rewriting the skill artifact.

A. Skill Revision

Skill revision is the content-changing stage of evolution: feedback modifies a persistent skill object, and the system determines whether the modification should survive. EvoSkill [92] fits this pattern because failed executions trigger a decision about whether to refine an existing skill or create a missing one. The accepted result is materialized as a structured skill folder with trigger metadata, instructions, and helper scripts. The held-out validation step is what makes this evolution rather than unconstrained self-editing: a candidate folder must improve future performance before it is retained.

Deployment-time revision adds a stronger requirement: the skill must be rewritten without corrupting the future library. Memento-Skills [82] addresses this by reading a relevant skill folder, executing under that guidance, and attributing failure before rewriting the prompt or program that will be reused later. Its unit-test gate and rollback step make revision reversible. The update is therefore not merely memory read and write, but a sequence of execution, attribution, rewriting, validation, and later reuse.

Longitudinal revision emphasizes identity across updates. AutoSkill [80] represents recurring behaviors as editable

SKILL.md artifacts and updates them through add, merge, or discard decisions when user feedback refines an existing capability. XSkill [93] extends this idea to multimodal agents by maintaining a skill document alongside experience records; its skill manager can update, merge, remove, or refine skill content as visually grounded rollouts and usage history accumulate. In both cases, the evolving unit is not a new trace but a persistent skill whose content is consolidated across later use.

The common claim is therefore narrow but strong. These systems demonstrate skill evolution when feedback changes a named, reusable artifact and when that artifact remains available after the update. They do not require the survey to treat every trajectory-driven skill construction method as evolution; the evidence lies in artifact revision, versioning, validation, and later reuse.

B. Skill Validation

Skill validation turns evolution into a survival problem: a revised skill must pass a check before it is trusted as future capability. SkillWeaver [56] constructs web-agent skills as APIs and hones them through practice, generated tests, and documentation updates—useful evidence, though weaker than full semantic verification since passing tests may only show that a function avoids immediate execution failure. Agent Skill Induction (ASI) [45] sharpens this check by treating executability as the boundary between acquisition and evolution: induced programs enter the refinement cycle only when verified against test trajectories, and prior skills can be updated when websites change. CoEvoSkills [96] targets richer multi-file skill packages and couples a skill generator with a surrogate verifier, so verification feedback itself co-evolves with the skill without access to ground-truth test content. TroVE [94]

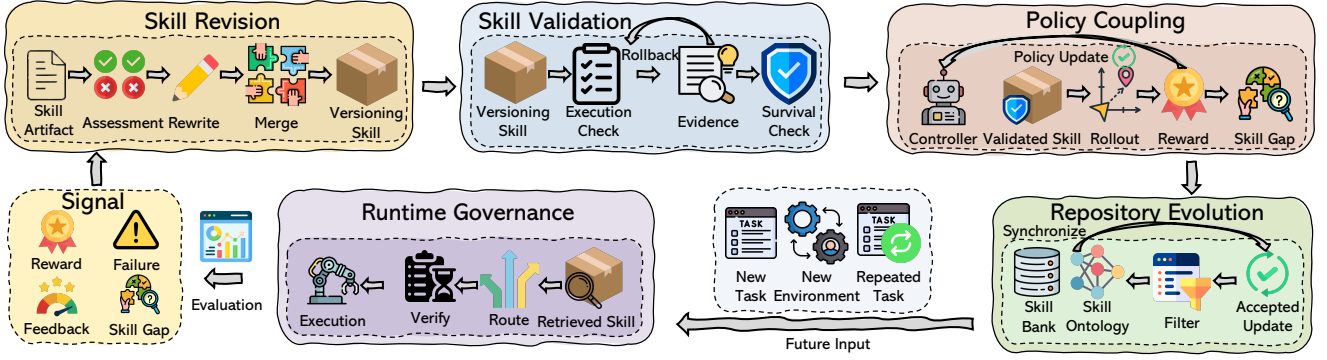


Fig. 9: Skill evolution through staged refinement: updates revise skills, validation filters changes, and trusted skills are indexed, retrieved, executed, and further improved.

extends the unit of repair from one skill to a function toolbox, growing and trimming entries via execution agreement, while Programmatic Skill Networks (PSN) [53] push further to a graph of executable symbolic skills with failure localization, maturity gates, and rollback-validated refactoring. Audited Skill-Graph [95] adds an evidence layer by promoting candidate skills into a directed graph only when supported by verifier reports and replayable evidence bundles. Across these methods, executable skill evolution depends on a survival check—execution, agreement, replay, or rollback—before a proposed update becomes part of the reusable substrate.

C. Policy Coupling

Policy coupling treats the skill substrate as part of the controller’s training state. SkillRL [84] builds a hierarchical SkillBank, retrieves general and task-specific skills during rollouts, and performs recursive skill evolution during reinforcement learning. Validation failures expose regions where existing skills are insufficient, while reward-driven policy updates change which failures and skill gaps the system encounters next. The skill bank is therefore a dynamic training component rather than a static context cache.

ARISE [97] makes the coupling explicit through a hierarchical manager–worker design. The skills manager selects relevant skills before execution and summarizes successful traces into a tiered cache–reservoir library after execution. Hierarchical rewards encourage both task success and useful skill use, while confidence and validation checks control whether generated or updated skills are inserted. The library is part of the agent state: selection, generation, addition, update, eviction, loading, and deletion co-adapt with the policy.

This stage is narrower than general continual learning. A method belongs here only when policy optimization changes the skill substrate and the changed substrate later affects the policy’s action space or rollout distribution. This is why CASCADE [107] is treated later as a boundary case: it provides useful evidence about cumulative executable scientific routines, but its central mechanism is continuous learning and self-reflection for scientific tool use rather than RL-style co-optimization of a skill library and controller.

D. Repository Evolution

Repository evolution asks how accepted skill changes scale beyond a single artifact. Uni-Skill [98] targets robotic manipulation settings where a fixed library cannot cover novel task decompositions. Its planner detects missing skills, requests supplementary skill descriptions, and uses SkillFolder to ground those descriptions in a hierarchical repository of annotated robotic video segments. The evolution claim should therefore be read as repository expansion and grounding, not as mature autonomous revision of existing entries.

The case for repository evolution is stronger when the repository itself is refined, filtered, and connected. SkillIX [99] treats a multi-level skill knowledge base as the object being improved: trajectories are distilled into planning, functional, and atomic skill levels; the resulting skills are refined and filtered; and coverage expands through experience-guided exploration. SkillNet [63] shifts the emphasis to shared infrastructure. It organizes a large repository through dynamic ontology construction, relation graphs, data-driven filtering, and multi-dimensional evaluation covering safety, completeness, executability, maintainability, and cost-awareness.

Collective evolution adds a synchronization problem. Skill-Claw [100] aggregates trajectories across users, lets an agentic evolver refine existing skills or create new ones, validates candidate updates in user environments, and synchronizes accepted changes back to the shared repository. This makes repository evolution depend on both update quality and propagation control. Local systems must avoid bad rewrites; repository systems must also avoid duplicate skills, inconsistent relations, weak coverage, and unsafe distribution. The output of this stage is therefore not an executed skill, but a searchable substrate from which runtime governance can retrieve candidates under a later task context.

E. Runtime Governance

An evolved skill changes behavior only if runtime control retrieves and uses it under the right conditions. Skill-Router [101] is therefore not a core revision method, but it exposes a necessary selection layer for large skill pools. Its retrieve-and-rerank pipeline shows that the full skill body provides stronger routing evidence than the name or description

TABLE IV: Representative methods that instantiate stages of skill refinement. Each row identifies the durable skill-side artifact, the trigger that updates it, the evolution operator, the validation check that controls survival, and the scope over which the evolved skill is reused.

Method	Evolving Artifact	Update Trigger	Evolution Operator	Validation Check	Reuse Scope
<i>Skill Revision</i>					
EvoSkill [92]	skill folder	failure trace	create, rewrite	held-out validation	cross-task
Memento-Skills [82]	skill folder	execution feedback	attribute, rewrite	unit-test rollback	cross-session
AutoSkill [80]	SKILL.md	user feedback + dialogue history	add, merge, discard	maintainer approval	personal
XSkill [93]	skill document + experience records	visual rollout + usage history	update, merge, remove, refine	skill-manager check	cross-task
<i>Skill Validation</i>					
SkillWeaver [56]	program API	practice outcome	debug, document, update	generated tests + debugging	cross-agent
ASI [45]	program skill	successful trajectories, website changes	induce, verify, update	test-trajectory execution	cross-task
CoEvoSkills [96]	multi-file skill package	surrogate-verifier feedback	generate, refine, co-evolve	surrogate verification	cross-task
TroVE [94]	function toolbox	streaming executions	grow, select, trim	execution agreement	cross-task
PSN [53]	skill graph	failure trace	repair, refactor	maturity gate + rollback validation	cross-task
Audited-SG [95]	skill graph	verifier report	verify, promote	replayable evidence bundle	ecosystem
<i>Policy Coupling</i>					
SkillRL [84]	SkillBank	RL reward + validation failures	generate, refine SkillBank	validation checkpoint	cross-task
ARISE [97]	tiered skill library	successful traces + hierarchical reward	select, generate, add, update, evict	confidence gate + skill validation	cross-task
<i>Repository Evolution</i>					
Uni-Skill [98]	SkillFolder repository	skill gap	expand, ground missing skills	video-aligned examples	cross-task
SkillX [99]	multi-level skill KB	execution feedback + exploration trace	refine, expand, update	skill filter	cross-task
SkillNet [63]	skill repository + ontology	repository contribution	create, evaluate, connect	repository rubric	ecosystem
SkillClaw [100]	shared skill repository	cross-user trajectory	refine, create, synchronize	environment validation + synchronization check	cross-user

Note: The table excludes experience-only, memory-only, one-shot acquisition, routing-only, and threat-analysis-only methods. **Update Trigger** denotes the signal that changes an existing skill substrate; **Evolution Operator** denotes the artifact-level operation; **Validation Check** denotes the evidence used to control survival, reuse, or ecosystem acceptance.

alone. Once a skill has been refined and indexed, the system still needs a selector that can retrieve the changed artifact rather than an obsolete or superficially similar one.

Governance addresses the corresponding failure mode: an evolved or shared skill may be executable, but not safe to trust at runtime. Audited Skill-Graph [95] represents the positive side of this boundary by tying promotion to replayable evidence. PoisonedSkills [102] represents the negative side: third-party skill documentation can hide malicious logic that agents later execute as trusted operational guidance. SkillClaw [100] similarly shows that collective evolution requires validation before synchronized updates are propagated to users.

Runtime governance closes the staged cycle by deciding whether a stored update actually affects future behavior. Revision changes a skill, validation decides whether the change should survive, and policy or repository mechanisms propagate the result. Runtime governance then retrieves, routes, trust-checks, executes, or retires the candidate. Execution produces the next reward, failure, feedback, skill gap, or trust signal. Provenance, permission boundaries, contamination detection, and retirement are needed so that an evolving ecosystem does not simply accumulate more executable risk.

F. Discussion

The evidence for skill evolution is strongest when a method directly modifies a named artifact and exposes a check on the update—revision methods such as EvoSkill [92], Memento-Skills [82], AutoSkill [80], and XSkill [93]; validation methods such as SkillWeaver [56], ASI [45], CoEvoSkills [96], TroVE [94], and PSN [53]; and substrate-level methods such as SkillRL [84], ARISE [97], SkillX [99], and SkillClaw [100]—whose common denominator is that a durable substrate is changed and later reused under a survival condition. Repository-scale autonomy is more partial: Uni-Skill [98], SkillNet [63], and Audited Skill-Graph [95] make repository evolution inspectable and governable but do not settle long-horizon causal attribution once many skills, routers, evaluators, and users interact. Boundary cases sharpen the scope: CASCADE [107] demonstrates cumulative skill growth without artifact-level revision; SkillRouter [101] and PoisonedSkills [102] show that later selection and trust can determine whether evolution helps or harms deployment; and SKILL0 [108] trades external governance for runtime efficiency by internalizing skills through an in-context RL curriculum.

VII. OPEN CHALLENGES

Despite the well-established lifecycle of agent skills, several critical challenges remain open across its various stages.

A. Skill Acquisition

Abstraction quality. Experience-derived acquisition must decide what to preserve from noisy trajectories and what to discard. Skills that remain too local behave like episodic memories; skills that over-abstract lose operational value.

Weak trigger specification. Many acquisition pipelines produce a plausible procedure but weak conditions for when it should be used. As a result, the skill itself may be useful, yet still fail in deployment because it is routed poorly.

Resource drift. As libraries mature, attached scripts, references, and schemas can become stale or inconsistent with the main document. Acquisition is therefore not a one-shot step—it creates artifacts that later sections must retrieve, evolve, and maintain.

Admission quality at scale. Faster acquisition pipelines can generate candidate skills more quickly than libraries can validate and curate them. This creates a familiar form of technical debt: low-quality skills accumulate, retrieval becomes noisier, and orchestration quality deteriorates.

B. Skill Retrieval

The current literature already offers several retrieval and selection strategies, but the central difficulties now lie less in defining isolated method categories than in making skill use robust at scale. These open challenges arise because skills are executable, structured, and continuously evolving artifacts rather than passive knowledge snippets.

Scalable skill libraries. Retrieval and selection must remain robust as skills are added, merged, revised, or deprecated over time. Existing systems make this challenge apparent, yet they still lack a general mechanism for keeping indexes, metadata, and priorities consistently synchronized as the skill library evolves [57], [63], [80], [82].

Constraint-aware composition. The problem is not only to find relevant skills, but to find skills that can be connected into a feasible execution path. Current work handles pieces of this through prerequisites, workflow memories, or compositional modules, but these constraints are still modeled in system-specific ways rather than through a shared abstraction [26], [45], [46], [56], [84].

Multi-objective selection. Behavioral success and utility can diverge from surface relevance, yet the field still lacks a common way to combine success, cost, latency, safety, risk, and user preference into one decision objective [81], [82], [90]. Many systems therefore still optimize a narrow proxy while leaving the rest of the decision problem implicit.

Execution-centric evaluation. Retrieval recall or selection accuracy cannot show whether the final decision improved end-to-end execution, saved cost, or introduced negative downstream effects [90]. Stronger evaluations should connect candidate quality, final decision quality, execution success, efficiency, recovery, and long-term utility in a single protocol.

Personalized selection. The same task can call for different skills under different user histories, habits, constraints, or preferences. Early personalized skill-bank systems suggest the promise of this direction, but personalization has not yet become a first-class part of the selection objective [80].

Adaptive selection and recovery. In real environments, skill use is a continuing control process: agents must revise choices after failures, switch to substitutes, and replan under updated states [81], [82], [84]. Thus, retrieval, selection, execution, and library evolution should be jointly modeled rather than treated as isolated steps. Future progress will rely not only on better similarity matching, but also on scalable skill management, structured constraints, richer objectives, and execution-centered evaluation.

C. Skill Evolution

Coarse artifact-level evaluation. Final task success cannot reveal whether a promoted artifact was actually reused, transferred across tasks, or selected for the right reason. TRACE [109] and STULIFE [71] push toward lifecycle behavior rather than endpoint accuracy, while MIRAGE [110] warns that correct answers do not establish that the underlying mechanism was appropriate. Most of the literature still reports outcome gains without isolating the causal contribution of the evolving artifact.

Asymmetric revision. Current systems are far better at adding artifacts than at safely rewriting or retiring them. EvolveR [111] explicitly scores, merges, and prunes principles; Memento-Skills [82] makes rollback and write gating unavoidable once the skill library is writable; MemEvolve [112] shows the asymmetry one layer deeper, where better content cannot compensate for an inadequate container. Growth is better understood than cleanup.

Weakly specified repository-scale governance. Once artifacts become shareable, the question shifts from *what to retain* to *who can publish, trust, deprecate, and bear responsibility* when an artifact is wrong. SkillNet [63] and SkillOS [65] address asset management; SkillRouter [101] shows that routing is itself a control bottleneck; skills.vote [113], Audited Skill-Graph [95], PoisonedSkills [102], and Anthropic Agent Skills [114] expose the pressures of discoverability, provenance, contamination, and externally managed ecosystem trust.

Confounded gains and long-horizon trust. Reported improvements may stem not from better artifacts but from stronger judges, better task synthesis, richer grounding, or more test-time compute—as visible in AgentEvolver [70], Explorer [115], and ReasoningBank [25], with the UI data-scale study [116] showing that scale alone can move metrics substantially. Writable memories, bridge objects, and skill repositories further introduce long-horizon trust requirements that current evaluations only partly address [20], [26], [64], [117]–[119]. Robust self-evolution therefore depends on trustworthy lineage, interpretable revision, and causal evaluation, not on persistence and reuse alone.

VIII. FUTURE RESEARCH DIRECTIONS

We outline five directions for advancing agent skill research.

Unified Skill Schema. Despite broad adoption of the skill abstraction, there is substantial heterogeneity in how $\mathcal{M}$ is written and how auxiliary resources are linked and maintained. A standardized schema defining common fields for scope, triggering conditions, dependencies, versioning, and safety constraints would make skills easier to share, retrieve, and govern across ecosystems.

Resource-Aware Joint Optimization. Current systems treat retrieval, planning, and execution as isolated stages [4], [15]. At scale, inference latency, token costs, and tool invocation risk become first-class constraints [13]. We should pursue end-to-end optimization that navigates the trade-off between utility, latency, and execution cost, including dynamic model routing and workload-aware scheduling [6], [18].

Skill Library Evolution under Non-Stationarity. APIs deprecate, tool behavior shifts, and task distributions change over time [10], [22]. Skill libraries need lifecycle-level robustness: drift detection, compatibility checks, safe online updates, and versioned rollback. Benchmarks should measure post-deployment stability and regression recovery, not only zero-shot success [120].

Multimodal and Domain-Specific Benchmarks. Current benchmarks are concentrated in text-centric settings. Practical deployments increasingly require perception-action integration under domain constraints. Embodied agents, autonomous driving, and low-altitude UAV scenarios are representative targets, where skills must be evaluated for safety, latency, and long-horizon decision quality.

Causality-Driven Skill Diagnosis. Many failures span retrieval, selection, and execution and cannot be resolved from surface-level logs [19], [121]. Systems should attribute failures to specific causes—retrieval mismatch, policy mis-selection, tool malfunction, or unsafe composition—and trigger targeted repair. This requires trace-level causal telemetry that connects runtime decisions to post-hoc diagnosis and safe updates [18].

IX. APPLICATION SCENARIOS

Agent skills manifest differently depending on domain demands. We organize representative scenarios into eight categories and highlight dominant skill formats and acquisition strategies. Fig. 10 illustrates these scenarios.

In software engineering, skills package recurring workflows such as code generation, debugging, and repository-aware refactoring so agents can apply verified routines instead of solving each issue from scratch [48]–[52], [94]. In web and GUI tasks, skills encode multi-step interaction routines over dynamic interfaces, supporting navigation, form completion, and recovery from UI changes [26], [46], [55], [56], [74], [76]. In chatbot systems, skills stabilize long-horizon dialogue by encoding memory-update policies, tool-routing routines, and failure-recovery procedures [5], [33], [34], [38], [40], [88]. In robotics, skills serve as reusable control routines connecting perception, action, and reward optimization, enabling agents to compose and adapt motor behaviors across tasks [28], [44], [47], [98]. In finance, skills encode reusable decision heuristics for analysis, planning, and portfolio adjustment under changing market conditions [37]. In healthcare, skills

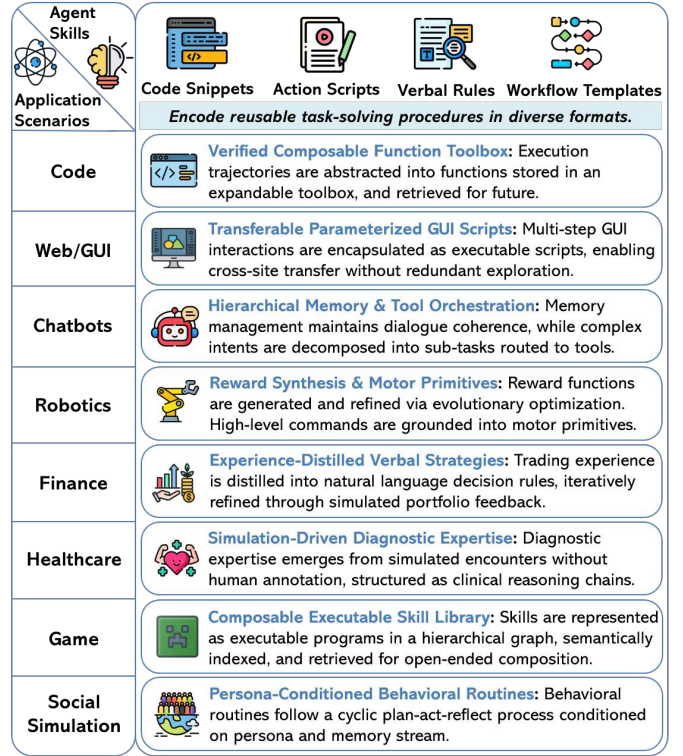


Fig. 10: Application scenarios of agent skills.

structure diagnostic reasoning and treatment planning into consistent procedures for medical decision support [66]. In game environments, skills are composable behavioral units discovered through continual interaction, enabling agents to build and reuse complex action repertoires during long-horizon tasks [12], [29], [30], [54]. In social simulation, skills encode reusable social-behavior routines that coordinate planning, acting, and reflection under constraints [20], [60].

X. RELATED WORK

This section reviews neighboring research streams and clarifies how our survey positions itself relative to prior literature.

• **Tool Use in LLM Agents.** A broad line of work studies how LLMs invoke external tools, APIs, databases, and execution environments to improve factuality and actionability [4], [5], [21], [22]. Early systems are largely API-centric, focusing on call planning and execution in prompted reasoning-action loops [4], [19], [122], [123]. As tool spaces expand, *tool retrieval* becomes a first-class problem: systems must map open-ended user intents to the right tool among large candidate sets, with growing attention to query-tool alignment, retrieval quality, and retrieve-then-call integration [83], [124]–[128]. The infrastructure narrative then shifts from ad hoc API wiring to protocol-guided interoperability: function-calling schemas and MCP-style interfaces separate reasoning from capability serving, enabling standardized access control, schema mediation, and cross-provider composition [10], [11], [105], [129].

• **RAG and Agent Memory.** Retrieval-centric research provides core mechanisms for non-parametric grounding, from dense/sparse retrieval and classic RAG pipelines to agentic retrieval under task constraints [15]–[17], [130]. More recent

lines also move beyond flat text retrieval toward structure-aware and graph-based retrieval settings, where document hierarchy, dependency structure, or graph organization influences candidate construction and reranking [60], [61], [131]–[133]. In parallel, memory-oriented work studies how agents preserve and reactivate long-horizon experience through memory management and consolidation [33], [34], [40], [41]. System-level memory architectures and recent long-term memory systems organize experience for multi-step agent coordination and extended-context use [60], [67]–[69], [134]. Complementary benchmarks evaluate such memory capabilities through test-time adaptation and memory quality [135]. Although closely related and discussed in externalization-oriented syntheses [105], these works center on retrieval and memory state, whereas our survey examines reusable procedural skills and their lifecycle in agent ecosystems.

XI. CONCLUSION

This survey examines LLM-based agent systems through the lens of agent skills—reusable procedural artifacts that coordinate tools, memory, and runtime context. While agents handle high-level reasoning and planning, skills from the operational layer enable reliable, composable execution. We organize the literature around four lifecycle stages: representation, acquisition, retrieval and selection, and evolution, offering a unified account of how procedural knowledge is externalized, activated, and maintained. Treating skills as first-class building blocks, rather than incidental prompts or tool wrappers, is key to improving the scalability, robustness, and governability of agent systems. We hope this framework provides a useful foundation for future research on agent skill systems, their supporting infrastructures, and the long-term management of reusable machine capabilities.

REFERENCES

- [1] T. B. Brown *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [2] J. Achiam *et al.*, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [3] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35. Curran Associates, Inc., 2022, pp. 27 730–27 744. [Online]. Available: <https://arxiv.org/abs/2203.02155>
- [4] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [5] Y. Shen *et al.*, “HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023. [Online]. Available: <https://arxiv.org/abs/2303.17580>
- [6] S. Hong *et al.*, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.00352>
- [7] OpenClaw, “Openclaw — the open-source personal ai assistant and autonomous agent,” <https://open-claw.org/>, 2026, official website, accessed April 21, 2026.
- [8] Manus, “Welcome - manus documentation,” <https://manus.im/docs>, 2026, official documentation, accessed April 21, 2026.
- [9] Anthropic, “Claude code overview,” <https://docs.anthropic.com/en/docs/claude-code/overview>, 2026, official documentation, accessed April 21, 2026.
- [10] —, “Introducing the model context protocol,” <https://www.anthropic.com/news/model-context-protocol>, 2024, anthropic Blog, November 2024.
- [11] OpenAI, “Function calling and other API updates,” <https://openai.com/blog/function-calling-and-other-api-updates>, 2023, openAI Blog, June 2023.
- [12] G. Wang *et al.*, “Voyager: An open-ended embodied agent with large language models,” *arXiv preprint arXiv:2305.16291*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.16291>
- [13] T. Cai *et al.*, “Large language models as tool makers,” in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2305.17126>
- [14] C. Qian *et al.*, “CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, 2023, pp. 6922–6939. [Online]. Available: <https://aclanthology.org/2023.findings-emnlp.462/>
- [15] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [16] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, 2020, pp. 6769–6781. [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>
- [17] Y. Du *et al.*, “AnyTool: Self-reflective, hierarchical agents for large-scale API calls,” *arXiv preprint arXiv:2402.04253*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.04253>
- [18] Q. Wu *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: <https://arxiv.org/abs/2308.08155>
- [19] N. Shinn *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023. [Online]. Available: <https://arxiv.org/abs/2303.11366>
- [20] J. S. Park *et al.*, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*. ACM, 2023, pp. 1–22. [Online]. Available: <https://arxiv.org/abs/2304.03442>
- [21] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023. [Online]. Available: <https://arxiv.org/abs/2302.04761>
- [22] Y. Qin *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” *arXiv preprint arXiv:2307.16789*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.16789>
- [23] A. Zhao *et al.*, “ExpeL: LLM agents are experiential learners,” in *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI)*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.10144>
- [24] X. Yang *et al.*, “Buffer of thoughts: Thought-augmented reasoning with large language models,” *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: <https://arxiv.org/abs/2406.04271>
- [25] S. Ouyang *et al.*, “Reasoningbank: Scaling agent self-evolving with reasoning memory,” *arXiv preprint arXiv:2509.25140*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.25140>
- [26] Z. Z. Wang *et al.*, “Agent workflow memory,” *arXiv preprint arXiv:2409.07429*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.07429>
- [27] J. Ni *et al.*, “Trace2skill: Distill trajectory-local lessons into transferable agent skills,” *arXiv preprint arXiv:2603.25158*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.25158>
- [28] M. Ahn *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” in *Conference on Robot Learning (CoRL)*, 2022. [Online]. Available: <https://arxiv.org/abs/2204.01691>
- [29] Z. Wang *et al.*, “Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.01560>
- [30] X. Zhu *et al.*, “Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory,” *arXiv preprint arXiv:2305.17144*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.17144>
- [31] S. Hao *et al.*, “Reasoning with language model is planning with world model,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 8154–8173.
- [32] W. Yao *et al.*, “Retroformer: Retrospective large language agents with policy gradient optimization,” *arXiv preprint arXiv:2308.02151*, 2023.

- [33] C. Packer *et al.*, “Memgpt: Towards LLMs as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.08560>
- [34] L. Liu *et al.*, “Think-in-memory: Recalling and post-thinking enable LLMs with long-term memory,” *arXiv preprint arXiv:2311.08719*, 2023. [Online]. Available: <https://arxiv.org/abs/2311.08719>
- [35] P. Zhou *et al.*, “Self-discover: Large language models self-compose reasoning structures,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 126 032–126 058, 2024.
- [36] M. Yuksekgonul *et al.*, “Optimizing generative ai by backpropagating language model feedback,” *Nature*, vol. 639, no. 8055, pp. 609–616, 2025.
- [37] Y. Yu *et al.*, “Fincon: A synthesized LLM multi-agent system with conceptual verbal reinforcement for enhanced financial decision making,” *arXiv preprint arXiv:2407.06567*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.06567>
- [38] Y. Wang *et al.*, “M4: Extending memoryllm with scalable long-term memory,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.00592>
- [39] J. Michelman *et al.*, “Enhancing reasoning with collaboration and memory,” *arXiv preprint arXiv:2503.05944*, 2025.
- [40] J. Nan *et al.*, “Memori: Self-organizing agent memory inspired by cognitive science,” *arXiv preprint arXiv:2508.03341*, 2025. [Online]. Available: <https://arxiv.org/abs/2508.03341v1>
- [41] a. others, “Intrinsic memory agents: Heterogeneous multi-agent llm systems through structured contextual memory,” *arXiv preprint arXiv:2506.19413*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.19413>
- [42] Q. Mi *et al.*, “Procmem: Learning reusable procedural memory from experience via non-parametric ppo for llm agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.01869>
- [43] S. Chen *et al.*, “Skillcraft: Can LLM agents learn to use tools skillfully?” *arXiv preprint arXiv:2603.00718*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.00718>
- [44] S. Yu, G. Li, W. Shi, and P. Qi, “Polyskill: Learning generalizable skills through polymorphic abstraction,” *International Conference on Learning Representations*, 2026. [Online]. Available: <https://arxiv.org/abs/2510.15863>
- [45] Z. Z. Wang *et al.*, “Inducing programmatic skills for agentic tasks,” 2025, published at COLM 2025. [Online]. Available: <https://arxiv.org/abs/2504.06821>
- [46] T. Chen *et al.*, “Cua-skill: Develop skills for computer using agent,” *arXiv preprint arXiv:2601.21123*, 2026.
- [47] Y. J. Ma *et al.*, “Eureka: Human-level reward design via coding large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.12931>
- [48] X. Yue *et al.*, “Ds-agent: Automated data science by empowering large language models with case-based reasoning,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.17453>
- [49] X. Zhong *et al.*, “Debug like a human: A large language model debugger via verifying runtime execution step-by-step,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.16906>
- [50] X. Wang *et al.*, “Executable code actions elicit better LLM agents,” *arXiv preprint arXiv:2402.01030*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01030>
- [51] J. Yang *et al.*, “Swe-agent: Agent-computer interfaces enable automated software engineering,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>
- [52] K. Zhang *et al.*, “Toolcoder: Teach code generation models to use api search tools,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.04032>
- [53] H. Shi *et al.*, “Evolving programmatic skill networks,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.03509>
- [54] Z. Wang *et al.*, “Jarvis-1: Open-world multi-task agents with memory-augmented multimodal language models,” *arXiv preprint arXiv:2311.05997*, 2023. [Online]. Available: <https://arxiv.org/abs/2311.05997>
- [55] L. Zheng *et al.*, “Synapse: Trajectory-as-exemplar prompting with memory for computer control,” *arXiv preprint arXiv:2306.07863*, 2024. [Online]. Available: <https://arxiv.org/abs/2306.07863>
- [56] B. Zheng *et al.*, “Skillweaver: Web agents can self-improve by discovering and honing skills,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.07079>
- [57] H. Li *et al.*, “Organizing, orchestrating, and benchmarking agent skills at ecosystem scale,” *arXiv preprint arXiv:2603.02176*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.02176>
- [58] J. Ruan *et al.*, “Tptu: large language model-based ai agents for task planning and tool usage,” *arXiv preprint arXiv:2308.03427*, 2023.
- [59] K. Christakopoulou *et al.*, “Agents thinking fast and slow: A talker-reasoner architecture,” *arXiv preprint arXiv:2410.08328*, 2024.
- [60] H. Yang, J. Chen, M. Siew, T. Llorido-Bofran, and C. Joe-Wong, “Llm-powered decentralized generative agents with adaptive hierarchical knowledge graph for cooperative planning,” *arXiv preprint arXiv:2502.05453*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.05453>
- [61] F. Wang *et al.*, “Graphskill: Documentation-guided hierarchical retrieval-augmented coding for complex graph reasoning,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.06620>
- [62] J. Qiu *et al.*, “Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution,” *arXiv preprint arXiv:2505.20286*, 2025.
- [63] Y. Liang *et al.*, “Skillnet: Create, evaluate, and connect ai skills,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.04448>
- [64] Y. Jiang *et al.*, “Sok: Agentic skills – beyond tool use in llm agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.20867>
- [65] L. Chen *et al.*, “Skills are the new apps – now it’s time for skill os,” 2026, preprints.org manuscript 202602.1096.v1. [Online]. Available: <https://www.preprints.org/manuscript/202602.1096/v1>
- [66] J. Li *et al.*, “Agent hospital: A simulacrum of hospital with evolvable medical agents,” *arXiv preprint arXiv:2405.02957*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.02957>
- [67] C. Hu *et al.*, “Evermemos: A self-organizing memory operating system for structured long-horizon reasoning,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.02163>
- [68] L. Yue *et al.*, “Hypermem: Hypergraph memory for long-term conversations,” 2026, accepted to ACL 2026 Main. [Online]. Available: <https://arxiv.org/abs/2604.08256>
- [69] G. Zhang *et al.*, “G-memory: Tracing hierarchical memory for multi-agent systems, 2025,” URL <https://arxiv.org/abs/2506.07398>.
- [70] Y. Zhai, S. Tao, C. Chen, A. Zou, Z. Chen, Q. Fu, S. Mai, L. Yu, J. Deng, Z. Cao *et al.*, “Agentevolver: Towards efficient self-evolving agent system,” *arXiv preprint arXiv:2511.10395*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.10395>
- [71] Y. Cai *et al.*, “Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.19005>
- [72] L. Qiu *et al.*, “Autorefine: From trajectories to reusable expertise for continual llm agent refinement,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.22758>
- [73] W. Tan *et al.*, “Cradle: Empowering foundation agents towards general computer control,” *arXiv preprint arXiv:2403.03186*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.03186>
- [74] C. Zhang *et al.*, “Appagent: Multimodal agents as smartphone users,” *arXiv preprint arXiv:2312.13771*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.13771>
- [75] Y. Fu *et al.*, “Autoguide: Automated generation and selection of context-aware guidelines for large language model agents,” *arXiv preprint arXiv:2403.08978*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.08978>
- [76] S. Zhou *et al.*, “WebArena: A realistic web environment for building autonomous agents,” in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2307.13854>
- [77] Y. Sun *et al.*, “Don’t retrieve, navigate: Distilling enterprise knowledge into navigable agent skills for qa and rag,” *arXiv preprint arXiv:2604.14572*, Apr. 2026. [Online]. Available: <https://arxiv.org/abs/2604.14572>
- [78] J. Qiu *et al.*, “Agentdistill: Training-free agent distillation with generalizable mcp boxes,” *arXiv preprint arXiv:2506.14728*, 2025.
- [79] J. Wang *et al.*, “Reinforcement learning for self-improving agent with skill library,” *arXiv preprint arXiv:2512.17102*, 2025.
- [80] Y. Yang *et al.*, “Autoskill: Experience-driven lifelong learning via skill self-evolution,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.01145>
- [81] H. Zhang *et al.*, “Memskill: Learning and evolving memory skills for self-evolving agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.02474>
- [82] Memento-Team, “Memento-skills: Let agents design agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.18743>
- [83] R. Wang *et al.*, “Toolgen: Unified tool retrieval and calling via generation,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [84] P. Xia *et al.*, “Skillrl: Evolving agents via recursive skill-augmented reinforcement learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.08234>

- [85] Y. Liu *et al.*, “Tool-planner: Task planning with clusters across multiple tools,” *arXiv preprint arXiv:2406.03807*, 2024.
- [86] Z. Zhang *et al.*, “Toolexpnet: Optimizing multi-tool selection in llms with similarity and dependency-aware experience networks,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 15 706–15 722.
- [87] Y. Xiao *et al.*, “Toolmem: Enhancing multimodal agents with learnable tool capability memory,” *arXiv preprint arXiv:2510.06664*, 2025.
- [88] K. Yang *et al.*, “Plugmem: A task-agnostic plugin memory module for llm agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.03296>
- [89] J. Wang *et al.*, “Skillorchestra: Learning to route agents via skill transfer,” *arXiv preprint arXiv:2602.19672*, 2026.
- [90] X. Li *et al.*, “Skillsbench: Benchmarking how well agent skills work across diverse tasks,” *arXiv preprint arXiv:2602.12670*, 2026.
- [91] C. Qian *et al.*, “Smart: Self-aware agent for tool overuse mitigation,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4604–4621.
- [92] S. Alzubi *et al.*, “Evoskill: Automated skill discovery for multi-agent systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.02766>
- [93] G. Jiang *et al.*, “Xskill: Continual learning from experience and skills in multimodal agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.12056>
- [94] Z. Wang *et al.*, “Trove: Inducing verifiable and efficient toolboxes for solving programmatic tasks,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.12869>
- [95] K. Huang and J. Huang, “Audited skill-graph self-improvement for agentic llms via verifiable rewards, experience synthesis, and continual memory,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.23760>
- [96] H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng, X. Liu, X. Li, and P. S. Yu, “CoEvoSkills: Self-evolving agent skills via co-evolutionary verification,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.01687>
- [97] Y. Li *et al.*, “Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.16060>
- [98] S. Xie *et al.*, “Uni-skill: Building self-evolving skill repository for generalizable robotic manipulation,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.02623>
- [99] C. Wang *et al.*, “Skillx: Automatically constructing skill knowledge bases for agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.04804>
- [100] Z. Ma *et al.*, “Skillclaw: Let skills evolve collectively with agentic evolver,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.08377>
- [101] Y. Zheng *et al.*, “Skillrouter: Retrieve-and-rerank skill selection for llm agents at scale,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.22455>
- [102] “Poisonedskills: Exploiting implicit trust in llm coding agent skill ecosystems,” 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.19281322>
- [103] Y. Qin *et al.*, “Tool learning with foundation models,” *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–40, 2024.
- [104] M. F. Chen *et al.*, “Skill-it! A data-driven skills framework for understanding and training language models,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023. [Online]. Available: <https://arxiv.org/abs/2307.14430>
- [105] C. Zhou *et al.*, “Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.08224>
- [106] W. Su, J. Long, Q. Ai, Y. Tang, C. Wang, Y. Tu, and Y. Liu, “Skill retrieval augmentation for agentic ai,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.24594>
- [107] X. Huang *et al.*, “Cascade: Cumulative agentic skill creation through autonomous development and evolution,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.23880>
- [108] Z. Lu *et al.*, “Skill0: In-context agentic reinforcement learning for skill internalization,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.02268>
- [109] D. Guo *et al.*, “Towards self-evolving benchmarks: Synthesizing agent trajectories via test-time exploration under validate-by-reproduce paradigm,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.00415>
- [110] J. Li *et al.*, “Mirage: Evaluating and explaining inductive reasoning process in language models,” 2025, published at ICLR 2025.
- [111] R. Wu *et al.*, “Evolver: Self-evolving llm agents through an experience-driven lifecycle,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.16079>
- [112] G. Zhang *et al.*, “Memevolve: Meta-evolution of agent memory systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.18746>
- [113] SkillsVote, “Skillsvote: The skills engine for ai agents,” 2026, accessed April 12, 2026. [Online]. Available: <https://skills.vote>
- [114] Anthropic, “Equipping agents for the real world with agent skills,” 2025, anthropic blog post dated October 16, 2025; used as packaged-skill ecosystem support evidence. [Online]. Available: <https://claude.com/blog/equipping-agents-for-the-real-world-with-agent-skills>
- [115] V. Pahuja *et al.*, “Explorer: Scaling exploration-driven web trajectory synthesis for multimodal web agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.11357>
- [116] W. Li *et al.*, “On the effects of data scale on ui control agents,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.03679>
- [117] Q. Zhang *et al.*, “Agentic context engineering: Evolving contexts for self-improving language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.04618>
- [118] P. Du, “Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.07670>
- [119] W. Huang *et al.*, “Rethinking memory mechanisms of foundation agents in the second half: A survey,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.06052>
- [120] Y. Shen *et al.*, “TaskBench: Benchmarking large language models for task automation,” in *Advances in Neural Information Processing Systems*, vol. 37. Curran Associates, Inc., 2024. [Online]. Available: <https://arxiv.org/abs/2311.18760>
- [121] C. S. Xia *et al.*, “Agentless: Demystifying LLM-based software engineering agents,” *arXiv preprint arXiv:2407.01489*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.01489>
- [122] L. Wang *et al.*, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. Toronto, Canada: Association for Computational Linguistics, 2023, pp. 2609–2634. [Online]. Available: <https://aclanthology.org/2023.acl-long.147/>
- [123] S. Yao *et al.*, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023. [Online]. Available: <https://arxiv.org/abs/2305.10601>
- [124] M. Li *et al.*, “Api-bank: A comprehensive benchmark for tool-augmented llms,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 3102–3116.
- [125] Z. Shi *et al.*, “Retrieval models aren’t tool-savvy: Benchmarking tool retrieval for large language models,” in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 24 497–24 524.
- [126] Y. Zhang *et al.*, “Data-efficient massive tool retrieval: A reinforcement learning approach for query-tool alignment with language models,” in *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, 2024, pp. 226–235.
- [127] W. Liu *et al.*, “Toolace: Winning the points of llm function calling,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [128] S. Yuan *et al.*, “Easytool: Enhancing llm-based agents with concise tool instruction,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 951–972.
- [129] H. Chase, “LangChain,” <https://github.com/langchain-ai/langchain>, 2022, open-source framework for building LLM-powered applications.
- [130] T. P. Ferraz *et al.*, “Retrieval-augmented LLM agents: Learning to learn from experience,” *arXiv preprint arXiv:2603.18272*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.18272>
- [131] Y. Zhou *et al.*, “In-depth analysis of graph-based rag in a unified framework,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.04338>
- [132] Y. Su *et al.*, “Clue-rag: Towards accurate and cost-efficient graph-based rag via multi-partite graph and query-driven iterative retrieval,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.08445>
- [133] F. Zhang *et al.*, “Erarag: Efficient and incremental retrieval augmented generation for growing corpora,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.20963>
- [134] Y. Chen *et al.*, “Msa: Memory sparse attention for efficient end-to-end memory model scaling to 100m tokens,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.23516>
- [135] T. Wei *et al.*, “Evo-memory: Benchmarking llm agent test-time learning with self-evolving memory,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.20857>